

Task 1 — Create a Production-Style VM for a Team Application

Real-World Scenario - Your team wants a Linux server on cloud where: developers can deploy applications , logs can be monitored , future backend services can run

Your job: Create a VM in GCP , Configure firewall access , Install a web server , Host a sample webpage. This is one of the most common real-world cloud tasks.

Solution :

- After naming your project & enabling compute engine API , Go to compute engine , create instance (VM).
- Give name , choose region & zone , and choose e2-micro (cheap-small-tier).
- Change bootdisk to Ubuntu & give size , allow HTTP traffic and click create.
- You'll see your VM running , click on SSH , run these commands
sudo apt update [checks latest version & update packages]
sudo apt install nginx -y [nginx is a webserver , used in prod systems/Kubernetes/microservices etc.]
sudo systemctl start nginx [start nginx , systemctl (LinuxServiceManager) is used to start/stop/check services]
sudo systemctl enable nginx [enable on boot]
sudo systemctl status nginx [check status whether it is running etc] , Press q to exit
- Go back to VM page , Scroll (horizontally) to right , and copy external IP
- Paste it on a new window , you should see , Welcome to nginx!
- Come back to SSH window , and run this
- cd /var/www/html [default web root , files here are saved by nginx]
- sudo nano index.html [for editing the page]
 - replace content with
<h1>Cloud Native Lab</h1>
<h2>Hello from GCP VM!</h2>
<p>Hosted using Compute Engine</p>
Save by Pressing : ctrl O then enter then ctrl X
- Refresh the browser window that you opened earlier [welcome to nginx one] , you'll see the updated content.

Basic terms :

SSH – Stands for SecureShell , used to remotely access Linux servers & manage prod systems.

Command Syntax – (command) (option) (target) , Eg : (sudo) (apt install) (nginx -y)

Sudo = Run as Admin , apt = Package manager , nginx = software name , -y = auto yes

Sudo – Stands for Super User Do. Normal user = employee , sudo = administrator permission.

Nginx – Pronounced as Engine-X

Port 22 = SSH , Port 80 = HTTP , Port 443 = HTTPS , Port 3306 = MySQL

/var/www/html – Instead of C:\ , Linux path use : /

/var = variable data , /www = web content , /html = website files

Welcome to nginx!

If you see this page, the nginx web server is successfully installed and working. Further configuration is required.

For online documentation and support please refer to nginx.org.
Commercial support is available at nginx.com.

Thank you for using nginx.

Cloud Native Lab

Hello from GCP VM!

Hosted using Compute Engine

Task 2 — Enterprise IAM & Access Management

Real-World Scenario - You joined a project team at Accenture. Different people need different access: Developers → manage applications , Security team → audit permissions , Interns → only view resources , DevOps engineers → manage infrastructure , Applications → access cloud services securely

Your job: Understand IAM , Create users/roles , Apply least privilege , Work with service accounts , Secure cloud resources properly

Solution :

- Open IAM Dashboard , IAM & Admin , Click on IAM
- Explore Permission of Roles – IAM & Admin then Roles , search Compute Admin
- Inside IAM & Admin , go to Service Account , Create Custom , Give Name , Desc , Role etc , Click Done.
- Attach with VM if needed , Go to VM Instances , you'll see Service Account Attached

[Note : In Pluralsight sandbox, some IAM operations such as assigning roles to service accounts may be restricted due to limited permissions in the managed lab environment. This is expected behavior and not a configuration mistake.]

Concepts :

IAM Answers : Who are you? What can you access? What actions can you perform?

Authentication = Providing who you are {Passwords / OTP / Fingerprints}

Authorization = What are you allowed to do {Eg : Read only / Delete / Create VM etc}

Role = Collection of Permission ,

Instead of assigning: compute.instances.create | compute.instances.delete compute.instances.start

Google groups them into: Compute Admin

Types : Basic Roles { Owner / Editor etc } , Predefined Roles { Compute Admin / Storage Admin etc } , Custom Roles

IAM Binding? Structure : MEMBER + ROLE = ACCESS

Example: john@company.com + Compute Viewer = Can view VMs. This attachment is called: IAM Binding

What is a Principal? A principal = identity. Can be: user , group , service account , domain

Principal Type	Example
User	max@gmail.com
Group	dev-team@company.com
Service Account	backend-app@project.iam.gserviceaccount.com

Service Account = Represents application/service.

Real Life Analogy - Employee badge vs robot badge. Humans use normal accounts. Applications use service accounts.

Why Needed? Applications need cloud access too.

Example: VM accessing storage bucket , app writing logs , Kubernetes accessing database

Instead of passwords: they use service accounts.

Task 3 — Cloud Storage + Static Website Hosting + Lifecycle Management

Real-World Scenario - Your company wants to host: a frontend website , images , PDFs , static assets

But they do NOT want: expensive VMs , server maintenance , OS management

So they decide to use: Cloud Storage Static Hosting

Your job: create storage bucket , upload website files , host static website , understand public/private access , configure lifecycle rules , learn storage architecture

Solution :

- ❖ Go to Cloud storage , click bucket , give name , region , storage class.
- ❖ Access Control : Uniform , also uncheck Public Access Prevention , Click create bucket
- ❖ Create a website file , say index.html , inside it , fill default information or any information.
- ❖ Upload this file into the bucket , Make object public
- ❖ Go to object tab , copy public url and open it , you should see your index.html in action
- ❖ Configure Lifecycle rule , go to lifecycle tab , click add rule , age : 30 days , Action : Set to nearline.

[Note : Cloud Storage objects are private by default. Public access requires explicit IAM permission such as storage.objects.get for allUsers. In sandbox environments, IAM modification permissions may be restricted, causing AccessDenied errors even if public access prevention is disabled.]

Concept :

Previously we did: website hosted on VM using Nginx , Now: website hosted WITHOUT server.

Cloud Storage Hosting : User - Cloud Storage Bucket - Static Files. { No server needed | Cheaper and simpler. }

Object Storage - Stores data as: objects , Each object contains: file data , metadata , unique identifier.

Static Website Hosting - Static website means: HTML , CSS , JS only . No backend processing.

Examples , Good for: portfolios , company landing pages , documentation , React frontend builds

Dynamic Website - Needs backend: login , database , API processing. Usually hosted differently.

Why VM Hosting Worked Earlier ?

Because: firewall rule allowed port 80 , nginx served files publicly. VM model : network-based exposure

Storage model: IAM-based object access

Task 4 — VPC Networking + Firewall Rules + Internal vs External Communication

Real-World Scenario - Your company has: frontend servers , backend APIs , databases

Security requirement: frontend should be public , backend should be private , database should NEVER be exposed to internet

Your job: understand cloud networking , create secure communication ,configure firewall rules , understand internal vs external traffic

Solution :

- Open VPC Dashboard , Go to VPC Networks , Click Default [Observe Subnet regions , IP ranges , rules etc]
- Create two VMs , One for Frontend and other for Backend , pick the same region for both.
- For Frontend allow HTTP traffic {Public} , and for backend , don't allow HTTP traffic {Private}.
- Under VPC , Go to Firewall [Observe existing rules & understand them]
- Go to VM dashboard , For frontend one , click SSH.
- Run these commands , ping backend-vm {if backvm name is backend-vm}
[This to test whether internal comm works , this works because firewall rule permit internal traffic]

[Note : Creating a VM only provisions infrastructure. Applications or websites require additional software such as web servers (e.g., Nginx) to actively listen for and respond to network requests.

Allowing firewall traffic does not guarantee application availability. A process/service must be actively listening on the target port.

A VM can be publicly reachable through networking configuration while still serving no content if no application is deployed.]

Concepts :

A VPC is your private network inside cloud. A subnet is smaller network inside VPC , subnets own IP ranges.

Default Network : GCP automatically creates: default VPC , default subnets , default firewall rules.

Helpful for beginners. Real companies usually create custom VPCs.

Firewall controls allowed network traffic.

Ingress - Traffic ENTERING resource. Example: browser accessing VM

Egress - Traffic LEAVING resource. Example: VM calling external API

default-allow-ssh : Allows: port 22 - for SSH.

default-allow-http: Allows: port 80 - for websites.

default-allow-internal: Allows internal VM communication.

Without firewall: everything exposed , huge security risk

Task 5 — Docker + Containerization + Container Registry

Real-World Scenario - Your development team says: “The application works on my laptop but fails on server.” Classic software problem. Different environments have: different OS versions , missing dependencies , conflicting libraries. To solve this, modern cloud systems use: containers

Your job: understand Docker , containerize an application , run containers , understand images , push to container registry

Solution :

- Create a VM , for machine type select , e2-med [Containers need slightly more resource] , allow HTTP.
- Click SSH , Run these commands :
sudo apt update
sudo apt install docker.io -y [Installing Docker , Ubuntu downloaded : DockerEngine , ContainerRuntime , RequiredPackages]
sudo systemctl start docker [Start Docker]
sudo systemctl enable docker [Enable on boot]
sudo systemctl status docker [Checking Docker Status , You should see active (running)]
sudo docker version [Shows docker client/server details , confirms installation]
sudo docker run hello-world [Running your first container]
sudo docker images [View Downloaded Images]
sudo docker run -d -p 80:80 nginx [Run real web container]
- Go to VM , copy external IP , you should see the web page running [Its running inside the container , not on VM]
- Run these commands :
sudo docker ps [viewing running containers]
sudo docker stop CONTAINER-ID [for stopping container , from above command , paste container ID on this]
sudo docker start CONTAINER-ID [for starting the container]
sudo docker ps -a [viewing all containers]
sudo docker rm CONTAINER-ID [for removing a container]
sudo docker rmi nginx [for removing an image]
ps aux | grep docker [for observing docker/daemon process]

Concepts :

A container is a packaged application with everything it needs. Including: code , runtime , libraries , dependencies , configs. Real life analogy : Container = lunchbox , Inside: food , spoon , napkin etc , Everything needed included. Works anywhere.

Without containers , App may fail because: server missing Java , wrong Python version , incompatible packages

With containers , Application behaves same everywhere like Developer laptop , Testing , Production etc.

Docker is the most popular container platform. It helps: create containers , run containers , distribute containers

Old Problem - Developers said works on my machine. Ops team: fails in production. Environment mismatch.

Docker Solution - Package: application + environment together

Virtual Machine , Contains: full OS , guest kernel , large size

Container , Shares host OS kernel. Much lighter.

Docker Engine - Core software running containers. Responsible for: creating containers , managing images , networking , storage

After checking status of docker , when it shows – active (running) , This Means , Docker daemon is now running.
Daemon - Background service/process. Runs continuously. Examples: docker daemon , nginx service , database service

After running our first container by this command : `sudo docker run hello-world` ,
Docker checked: Do I already have hello-world image? Likely: NO

Then: Docker Hub → Downloaded image → Created container → Ran application → Printed output → Container exited

Docker Hub - Public container registry , Like: Play Store for containers. Stores images.

Image = blueprint/template. Contains: app , dependencies , runtime.

Real Life Analogy , Image = cake recipe (template) , Container = actual baked cake (running instance)

After viewing downloaded images by this command - `sudo docker images` , You'll see: hello-world

What You're Seeing - Local image cache. Docker stores downloaded images locally.

`sudo docker run -d -p 80:80 nginx` , breaking down this command :

Part	Meaning
------	---------

<code>docker run</code>	create/run container
-------------------------	----------------------

<code>-d</code>	detached/background mode
-----------------	--------------------------

<code>-p 80:80</code>	port mapping
-----------------------	--------------

<code>nginx</code>	image name
--------------------	------------

Port Mapping - Meaning of 80:80 - HOST_PORT : CONTAINER_PORT

Real Meaning – Internet → VM Port 80 → Container Port 80 → Nginx inside container

Container has its own isolated networking. Port mapping exposes it externally.

What makes Container Powerful : Portable (runs anywhere) , Lightweight (smaller than VMs) , Fast Startup (In seconds) , Consistent (Same environment) , Scalable (Easy replication).

Container Registry - Registry stores container images. Examples:

Registry	Platform
----------	----------

Docker Hub	public
------------	--------

Artifact Registry	GCP
-------------------	-----

ECR	AWS
-----	-----

Real Workflow : Developer builds image → Pushes to registry → Production pulls image → Runs container

Artifact Registry - Modern GCP registry service. Stores: Docker images , packages , artifacts

Why Containers Matter for Kubernetes - Kubernetes does NOT run source code directly. It runs: containers

Detached Mode (-d) - Without -d : terminal stays attached , With -d : container runs in background.

Cloud-Native Systems Love Containers - Because they: scale easily , deploy consistently , work across environments
integrate with CI/CD

Sandbox Images/Screenshots : 1) Running inside a container , 2) After stopping container , the same web page stops

Welcome to nginx!

If you see this page, nginx is successfully installed and working. Further configuration is required for the web server, reverse proxy, API gateway, load balancer, content cache, or other features.

For online documentation and support please refer to nginx.org.
To engage with the community please visit community.nginx.org.
For enterprise grade support, professional services, additional security features and capabilities please refer to f5.com/nginx.

Thank you for using nginx.



This site can't be reached

34.63.46.11 refused to connect.

Try:

- Checking the connection
- Checking the proxy and the firewall

ERR_CONNECTION_REFUSED

[Reload](#) [Details](#)

3) Running Commands inside SSH

```
cloud_user_p_1a2ecf41@docker-vm:~$ sudo docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
4f55086f7dd0: Pull complete
d5e71e642bf5: Download complete
Digest: sha256:0e760fd9bc48ba8041e7c6db999bb40bfca508b4be580ac75d32c4e29d202ce1
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
   (amd64)
3. The Docker daemon created a new container from that image which runs the
   executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
   to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/

cloud_user_p_1a2ecf41@docker-vm:~$ sudo docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
hello-world:latest	0e760fd9bc48	25.9kB	9.49kB	U

```
cloud_user_p_1a2ecf41@docker-vm:~$ sudo docker run -d -p 80:80 nginx
Unable to find image 'nginx:latest' locally
latest: Pulling from library/nginx
388081780287: Pull complete
fa14110e5ccb: Pull complete
5b4d6ff92fc4: Pull complete
e2dc6cdcd988: Pull complete
cf902b7062fb: Pull complete
564041453b75: Pull complete
6735e617770d: Pull complete
9d69fba9fb0d: Download complete
d8c0b370a46a: Download complete
Digest: sha256:80067c98538c6bf725f5177e841aa720ae0ed1c378bbea368b6330bfe18a36b3
Status: Downloaded newer image for nginx:latest
d2ca3287b74b96a42f88312fc1d7d4c34de370022f1f2883edcb2ef040402536

cloud_user_p_1a2ecf41@docker-vm:~$ sudo docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS                               NAMES
d2ca3287b74b   nginx    "/docker-entrypoint..." About a minute ago Up About a minute 0.0.0.0:80->80/tcp, [::]:80->80/tcp youthful_hertz

cloud_user_p_1a2ecf41@docker-vm:~$ sudo docker stop CONTAINER-ID
Error response from daemon: No such container: CONTAINER-ID
cloud_user_p_1a2ecf41@docker-vm:~$ sudo docker stop ^C
cloud_user_p_1a2ecf41@docker-vm:~$ sudo docker stop d2ca3287b74b
d2ca3287b74b

cloud_user_p_1a2ecf41@docker-vm:~$ sudo docker start d2ca3287b74b
d2ca3287b74b

cloud_user_p_1a2ecf41@docker-vm:~$ sudo docker ps -a
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS                               NAMES
d2ca3287b74b   nginx    "/docker-entrypoint..." 4 minutes ago Up 47 seconds 0.0.0.0:80->80/tcp, [::]:80->80/tcp youthful_hertz
92bbf04d6be5   hello-world "/hello"                5 minutes ago Exited (0) 5 minutes ago modest_borg

cloud_user_p_1a2ecf41@docker-vm:~$ sudo docker rm 92bbf04d6be5
Error response from daemon: No such container: 92bbf04d6be5
cloud_user_p_1a2ecf41@docker-vm:~$ sudo docker rm 92bbf04d6be5
92bbf04d6be5

cloud_user_p_1a2ecf41@docker-vm:~$ ^C
cloud_user_p_1a2ecf41@docker-vm:~$ sudo docker ps -a
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS                               NAMES
d2ca3287b74b   nginx    "/docker-entrypoint..." 6 minutes ago Up 2 minutes 0.0.0.0:80->80/tcp, [::]:80->80/tcp youthful_hertz
```

Task 6 — Build Your Own Docker Image + Dockerfile + Artifact Registry

Real-World Scenario - In Task 5: you USED existing container images like nginx. But real cloud-native engineers usually: build custom images for their own applications. Example: Java backend , Node.js API , Python Flask app , React frontend

Your job: create your own application , write Dockerfile , build custom image , run container , understand image layers , understand registry workflows

Solution :

- Create a VM , e2-medium and allow HTTP. Click SSH.
- Install docker {sudo apt update , sudo apt install docker.io -y}
- Start docker {sudo systemctl start docker , sudo systemctl enable docker}
- Create Project Folder {mkdir myapp , cd myapp}
- Create HTML file , run nano index.html , paste this :

```
<!DOCTYPE html>
<html>
<head>
  <title>My Custom Docker App</title>
</head>
<body>
  <h1>Hello from Custom Docker Image!</h1>
  <h2>Built by Maxx</h2>
  <p>This application runs inside a custom container.</p>
</body>
</html>
```

Save it : Ctrl O + Enter + Ctrl X.
- Create docker file , run - nano Dockerfile. Paste this :
FROM nginx
COPY index.html /usr/share/nginx/html/index.html. [Save it , same step as above]
- Build Docker Image , run : sudo docker build -t my-custom-nginx . [' . ' means current directory]
- View Images , sudo docker images.
- Run Custom Container , sudo docker run -d -p 80:80 my-custom-nginx
- Copy External IP of the VM , you should see your custom web page.
- Come back to SSH , Inspect running containers , sudo docker ps. [Observe Image used , ports , IDs etc]
- Open Container Shell , sudo docker exec -it CONTAINER-ID bash. [Copy your container ID from prev step]
- Observe Files , ls /usr/share/nginx/html [you should see index.html]
- Type exit , to return to SSH.

Concepts :

Dockerfile is a recipe/instruction file for building Docker images. It tells Docker: what base image to use , what files to copy , what commands to run , how app should start. Eg : Start with Ubuntu - Install Python - Copy app files - Run application → Dockerfile automates this.

Image Building - When Docker reads Dockerfile , it creates a reusable imag. Result - You get: portable application package , same behavior everywhere

Breaking down this command , FROM nginx | COPY index.html /usr/share/nginx/html/index.html.

FROM nginx - Means : start using official nginx image as base

Real Meaning - Instead of building: Linux , nginx , configs from scratch you reuse existing image.

COPY Command - Means : copy my local file into container filesystem

Why This Path? Nginx serves website files from: /usr/share/nginx/html

You are customizing existing nginx image. This is VERY common in real companies.

Breaking down this command , sudo docker build -t my-custom-nginx .

Part	Meaning
docker build	build image
-t	tag/name image
my-custom-nginx	image name
.	current directory

Docker Layers - Each Dockerfile instruction creates: image layer.

Why Layers Matter , Benefits: caching , faster builds , smaller downloads

Real Example , If only HTML changes: Docker reuses nginx layer , rebuilds only changed layer

Efficient.

After running custom container , this happens : VM Port 80 → Container Port 80 → Custom nginx → Your HTML page

Container Filesystem - Containers have isolated filesystem. Your copied HTML exists: inside container , not directly on VM.

Image Tagging? Image names/tags help versioning.

Example: myapp:v1 | myapp:v2 | myapp:latest

Real Company Usage , Used for : rollback , deployments , version tracking

Security Understanding - Never store: passwords , API keys , secrets inside Docker images.

Why? Because images can be shared publicly.

Real Company Practice - Secrets handled using: Secret Manager , Kubernetes secrets , environment variables

Why Base Images Matter - Using trusted base images is critical.

Bad image: malware risk , vulnerabilities

Companies scan images for security issues.

Sandbox Screenshots :

Hello from Custom Docker Image!

Built by Maxx

This application runs inside a custom container.

```

cloud_user_p_8cc0845d@docker-build-vm:~$ sudo systemctl start docker
cloud_user_p_8cc0845d@docker-build-vm:~$ sudo systemctl enable docker
cloud_user_p_8cc0845d@docker-build-vm:~$ mkdir myapp
cloud_user_p_8cc0845d@docker-build-vm:~$ cd myapp
cloud_user_p_8cc0845d@docker-build-vm:~/myapp$ nano index.html
cloud_user_p_8cc0845d@docker-build-vm:~/myapp$ nano Dockerfile
cloud_user_p_8cc0845d@docker-build-vm:~/myapp$ sudo docker build -t my-custom-nginx .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
             Install the buildx component to build images with BuildKit:
             https://docs.docker.com/go/buildx/

Sending build context to Docker daemon 3.072kB
Step 1/2 : FROM nginx
latest: Pulling from library/nginx
388081780287: Pulling fs layer
e2dc6cdcd988: Pulling fs layer
564041453b75: Pulling fs layer
6735e617770d: Pulling fs layer
fa14110e5ccb: Pulling fs layer
5b4d6ff92fc4: Pulling fs layer
cf902b7062fb: Pulling fs layer
564041453b75: Download complete
e2dc6cdcd988: Download complete
fa14110e5ccb: Download complete
388081780287: Download complete
6735e617770d: Download complete
9d69fba9fb0d: Download complete
d8c0b370a46a: Download complete
5b4d6ff92fc4: Download complete
cf902b7062fb: Download complete
5b4d6ff92fc4: Pull complete
564041453b75: Pull complete
fa14110e5ccb: Pull complete
cf902b7062fb: Pull complete
e2dc6cdcd988: Pull complete
388081780287: Pull complete
6735e617770d: Pull complete
Digest: sha256:800e7c98538c6bf725f5177e841aa720ae0ed1c378bbea368b6330bfe18a36b3
Status: Downloaded newer image for nginx:latest
--> 800e7c98538c
Step 2/2 : COPY index.html /usr/share/nginx/html/index.html
--> 1ed13cb975b0
Successfully built 1ed13cb975b0
Successfully tagged my-custom-nginx:latest

```

```

cloud_user_p_8cc0845d@docker-build-vm:~/myapp$ sudo docker images

```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
my-custom-nginx:latest	1ed13cb975b0	238MB	63.1MB	
nginx:latest	800e7c98538c	241MB	66MB	

```

cloud_user_p_8cc0845d@docker-build-vm:~/myapp$ sudo docker run -d -p 80:80 my-custom-nginx
a9ade775509a4a76adb668b6c4282a58ada0c40375618ab0f1e8e73a219ec7a
cloud_user_p_8cc0845d@docker-build-vm:~/myapp$ sudo docker ps

```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
a9ade775509	my-custom-nginx	"/docker-entrypoint..."	48 seconds ago	Up 48 seconds	0.0.0.0:80->80/tcp, [::]:80->80/tcp	elated_elgamal

```

cloud_user_p_8cc0845d@docker-build-vm:~/myapp$ sudo docker exec -it CONTAINER-ID bash
Error response from daemon: No such container: CONTAINER-ID
cloud_user_p_8cc0845d@docker-build-vm:~/myapp$ sudo docker exec -it a9ade775509 bash
root@a9ade775509:/# ls /usr/share/nginx/html
50x.html  index.html
root@a9ade775509:/# exit
exit

```

Task 7 — Kubernetes Fundamentals + GKE + Deploying Containers at Scale

Real-World Scenario - Your company now has: hundreds of containers , multiple applications , traffic spikes , failures happening randomly. Managing containers manually becomes impossible. Example problems: container crashes at 3 AM , traffic suddenly increases , one VM dies , application needs scaling. This is why companies use: Kubernetes
Your job: understand orchestration , deploy containers at scale , manage replicas , expose applications , understand self-healing systems.

Solution :

- ❖ Go to Kubernetes Engine , enable API if prompted , Create Cluster , Choose Autopilot. Now open cloud shell.
Run these :
gcloud container clusters get-credentials my-first-gke-cluster --region asia-south1 [Connect kubectl to cluster]
kubectl get nodes [Check Nodes]
kubectl create deployment nginx-deployment --image=nginx [Deploy application]
kubectl get pods [View pods]
kubectl delete pod POD-NAME [deleting pods]
kubectl get pods [Pod comes back automatically, why – deployment said : desired state = 1 running pod]
kubectl scale deployment nginx-deployment --replicas=3 [Scale Deployment , now check pods – should see 3]
kubectl expose deployment nginx-deployment --type=LoadBalancer --port=80 [Expose Deployment]
kubectl get services [get external IP , open it , and deployment is complete]

[Note :

Managed Kubernetes services such as GKE automatically provision multiple underlying resources including node pools, worker nodes, networking, and orchestration infrastructure. In restricted sandbox environments, this may trigger quota or anti-abuse protections due to high infrastructure usage.

Creating a Kubernetes cluster provisions significantly more infrastructure than creating a single VM because Kubernetes requires orchestration, scaling, networking, and distributed workload management components.]

Concepts :

Docker solved: application packaging , But Kubernetes solves: large-scale container management

Kubernetes - Kubernetes is a container orchestration platform. “Orchestration” Means - Coordinating many moving parts automatically.

Real Life Analogy : Imagine restaurant kitchen. Without orchestration: chefs confused , duplicate work , missed orders

Kubernetes acts like: restaurant manager for containers

Kubernetes Handles - deployment , scaling , networking , recovery , scheduling , load balancing automatically.

Why Kubernetes Exists - Imagine manually managing: 1000 containers , across 50 servers. Impossible manually.

Kubernetes automates this.

GKE - Google Kubernetes Engine , Managed Kubernetes service on Google Cloud Platform. Google manages: control plane , upgrades , infrastructure complexity. You manage: applications

Cluster = group of machines working together | Node = VM running Kubernetes workloads

Worker Nodes Run: pods , containers , applications

Pod is smallest deployable unit in Kubernetes. Usually contains: one container , Sometimes: multiple tightly coupled containers

Kubernetes does NOT directly manage containers. It manages: pods

Real Life Analogy - Pod = apartment , Containers = people inside apartment.

Kubectl - Command-line tool for Kubernetes. Used to: deploy apps , inspect cluster , manage workloads

Declarative Infrastructure - Instead of manually creating things step-by-step , you describe: desired final state. Kubernetes ensures it happens. Example , You say: I want 3 replicas running , Kubernetes maintains that automatically.
Deployment - Deployment manages: pods , replicas , updates , recovery
Real Life Analogy - Deployment = supervisor ensuring enough workers exist.

Task 8 — Cloud Run + Serverless Containers + Modern Deployment

Real-World Scenario - Your company says: “We want containerized apps... but we do NOT want to manage: VMs
Kubernetes nodes , scaling , patching , infrastructure.” This is where: serverless containers become powerful.
Your job: deploy containerized application , without managing servers , using fully managed cloud services

Solution :

- Open Cloud run , Enable API , Create service , choose - Deploy one revision from existing container image.
- For container URL , type – nginx [Cloud Run will pull nginx container image , similar to Docker.]
- Give service name , region and allow public access. Click on Create the service
- Once deployed , cloud run will give an url , open it , you should see the webpage working.

[Note :

Cloud environments enforce quotas, limits, and governance policies to prevent excessive infrastructure provisioning and resource abuse.

Serverless platforms abstract infrastructure management but still provision underlying compute, memory, networking, and scaling resources.]

Concepts :

Cloud Run - Google Cloud Platform Cloud Run is serverless container platform. You provide container image , Google handles: servers , scaling , networking , infrastructure

Many modern companies now prefer: serverless-first architectures , because: simpler , cheaper less operational overhead.

Serverless - Serverless does NOT mean: servers don't exist , Servers DO exist. It means: YOU don't manage them , Cloud provider manages infrastructure. Real Life Analogy - Traditional hosting: renting and managing restaurant kitchen , Serverless: food delivery kitchen platform handles infrastructure , You focus only on: your application

Kubernetes - You manage: cluster , scaling rules , node pools , orchestration configs

Cloud Run - You just deploy container image. Google handles rest.

Revision - Every deployment creates new revision , Like application version snapshot. Benefits - rollback support , traffic splitting , version tracking

Cold Start - If Cloud Run scaled to zero: first request may take slightly longer. Because: container instance must start
This is called cold start

Task 9 — Cloud Logging + Monitoring + Troubleshooting (Sandbox Safe)

Real-World Scenario - A production application suddenly becomes: slow, unavailable, crashing, returning errors.

Manager asks: “What happened?” At this point, cloud engineers DO NOT: randomly restart infrastructure

First they: inspect logs, monitor metrics, identify root cause, troubleshoot systematically.

Solution :

- Create VM , give name etc , allow HTTP , choose Ubuntu. Click SSH
- Install Nginx [sudo apt update | sudo apt install nginx -y].
- Start Nginx [sudo systemctl start nginx] , Open external IP
- Explore logs directory [cd /var/log and then ls]
- View system logs [sudo tail /var/log/syslog]
- Live log monitoring [sudo tail -f /var/log/nginx/access.log]
- View Nginx error logs [sudo tail /var/log/nginx/error.log]

Concept :

Logging - recording system/application events. Examples: user login , server startup , errors , crashes , requests

Real Life Analogy - Logs are like: security camera recordings for systems. They help investigate: what happened, when, why.

Monitoring - continuously tracking system health. Examples: CPU usage , memory usage , network traffic , uptime

Observability = ability to understand system behavior , using: logs , metrics , traces.

Linux systems store logs inside: /var/log , Different services generate their own log files.

Syslog - Core Linux system logging mechanism. Tracks: system events , services , warnings , kernel messages

Tail - Command showing latest lines from file

Access Log – Records incoming requests, visitor IPs, request paths, response codes.

Error Log – Records failures, crashes, server errors.

Cloud Logging - Google Cloud Platform Cloud Logging collects VM logs , application logs , audit logs , infrastructure events , centrally.

Metrics are numerical measurements over time. Examples: CPU % , RAM usage , requests/sec

Logs vs Metrics - Logs: detailed events , Metrics: aggregated measurements.

Task 10 — CI/CD Pipeline + Cloud Build + Automated Container Deployment

Real-World Scenario - Imagine this. Developer pushes code to GitHub. Company DOES NOT want engineers manually: SSH into servers, build Docker images manually, and deploy containers manually.

Instead they want: push code → automatic build → automatic testing → automatic deployment. This is CI/CD.

Solution :

- Search Cloud Build , enable API , Go to trigger and click on create trigger , give name etc.
- For event type , if you have github repo , then select push to branch , otherwise select manual invocation.

For github , follow these :

- 1) Select push to branch [Means pipeline activates when dev pushes code into repo branch].
- 2) Source repo [Connect repo from github account]
- 3) For configuration , choose docker file [why – pipeline auto builds container img using docker instr]
- 4) For dockerfile location , use default i.e dockerfile.
- 5) Select Artifact destination , and click on create trigger

What Actually Happens Internally - When trigger activates:

Cloud Build provisions temporary build worker → Downloads source code → Reads Dockerfile → Builds container image → Pushes image to Artifact Registry → (Optional) deploys application.

For manual invocation , follow these :

- 1) Select manual invocation [Means : pipeline runs only when manually triggered | No GitHub required.]
- 2) For configuration , choose inline [Inline lets you write pipeline YAML directly inside trigger , Very useful for testing, learning, small pipelines.]
- 3) Edit YAML editor , and paste this :

steps:

```
- name: 'gcr.io/cloud-builders/docker'
  args: ['version']
```

[If above code gives error when running the trigger , add this at the end :

options:

```
logging: CLOUD_LOGGING_ONLY]
{meaning it stores logs only in Cloud Logging , instead of custom storage bucket}
What this pipeline does : runs docker version command inside Cloud Build worker.
```

- 4) Select a service account , Click create and run trigger.

What Happens Internally - Cloud Build will:

Create temporary build worker → start Docker builder container → execute docker version → store logs → destroy temporary environment.

OR in simple language : verify Docker exists in build environment.

Right now its too small to visibly impress , like in logs you'll see very minimal. To make it more visual , we'll upgrade the current pipeline , instead of docker version , we'll automate creation of files inside build pipeline.

- a) Replace the yaml code with this :

steps:

```
- name: 'ubuntu'
  entrypoint: 'bash'
  args:
    - '-c'
    - |
```

```
echo "Starting automated pipeline"
mkdir automation-demo
cd automation-demo
echo "Build completed successfully" > result.txt
cat result.txt
options:
logging: CLOUD_LOGGING_ONLY
```

What This Pipeline Does - When triggered:

Start temporary Linux worker → Create folder → Create file → Write output into file → Print file contents into logs → Destroy worker

b) Go to history , inside logs you'll see : Starting automated pipeline , Build completed successfully

[Note : While seeing logs in history , you might not able to see it due to IAM restrictions , you'll see like this :
Build execution encountered: IAM logging permission restriction , because sandbox service account lacked:
"logging.logEntries.create" permission.]

Concepts :

CI/CD - CI = Continuous Integration , Means: developers continuously merge code changes into shared repository.

Automated systems: build code, verify integration, detect issues early.

CD = Continuous Delivery / Deployment , Means: automated release pipeline that deploys applications safely and consistently.

Real Life Analogy - Think of factory assembly line. Instead of manually assembling each car: automation pipeline handles production flow

Cloud Build - Google Cloud Platform Cloud Build is managed CI/CD build service. It can build containers, run scripts, automate workflows, deploy applications.

Pipeline - sequence of automated steps. They are programmable automation workflows.

Trigger means event that starts pipeline automatically , Example: Git push , pull request , tag creation. It activates from external HTTP request/API call.

Immutable Deployment - Instead of modifying running servers , new version replaces old version

Build Artifact - Output produced by pipeline. Examples: Docker image , binary , deployment package

YAML - human-readable configuration language , Used heavily in Kubernetes , CI/CD , DevOps , Infrastructure-as-Code

Task 11 — Infrastructure as Code (IaC) with Terraform (VERY Important for Cloud-Native Roles)

Real-World Scenario - Imagine company has 200 VMs , 50 storage buckets , multiple VPCs , Kubernetes clusters , firewall rules . Manually creating everything from UI becomes impossible and error-prone. So companies define infrastructure using code. This is Infrastructure as Code (IaC).

Solution :

- ❖ Open cloud shell , usually located at top right corner.
- ❖ Run - terraform version [Checks whether Terraform CLI tool exists in environment]
- ❖ Run - mkdir terraform-demo | cd terraform-demo [Create working directory]
- ❖ Run - nano main.tf [Create Terraform config file] , paste this : {Replace project id with your actual project id}:

```
terraform {  
  required_providers {  
    google = {  
      source = "hashicorp/google"  
      version = "~> 5.0"  
    }  
  }  
}
```

```
provider "google" {  
  project = "PROJECT_ID"  
  region = "us-central1"  
}
```

```
resource "google_storage_bucket" "demo_bucket" {  
  name     = "maxx-terraform-demo-12345"  
  location = "US-CENTRAL1"  
}
```

Save it by : Ctrl O + Enter + Ctrl X

- ❖ Run : terraform init [Initialize terraform]
- ❖ Run : terraform validate [Validate configuration : checks whether code syntax is valid].
- ❖ Run : terraform plan [preview infrastructure changes]
- ❖ Run : terraform apply , and type yes [Apply Infrastructure].
- ❖ Your bucket should be created , verify it , go to cloud storage then buckets , your bucket should be there.
- ❖ Run : ls [Observe terraform state]
- ❖ Run : terraform destroy [Destroy Infrastructure that you just created]

Concepts :

Terraform - HashiCorp Terraform is Infrastructure as Code tool , used to provision cloud infrastructure, automate environments, manage resources declaratively. Terraform itself DOES NOT host infrastructure. It talks to cloud provider APIs , to create/manage resources.

Real Life Analogy - Terraform blueprint - "Build me this infrastructure layout automatically."

Terraform is declarative , Meaning you define: desired final state , NOT step-by-step commands.

Example - Instead of saying: Create VM , Attach disk , Configure network . you say: "I want 1 VM with this configuration."

Terraform figures out HOW. .tf files are Terraform configuration files

Provider = plugin Terraform uses to communicate with cloud APIs

Resource = actual infrastructure object , Examples: VM , bucket , network , firewall.

After initializing terraform , terraform - downloads provider plugins, initializes working directory, prepares environment. Terraform is modular. Providers are downloaded dynamically.

Terraform plan - Terraform shows what changes it intends to make BEFORE actual creation.

Terraform apply - Terraform now calls Google Cloud APIs automatically to provision infrastructure.

Terraform state file stores current infrastructure knowledge. Terraform compares: desired state vs actual state, using state tracking.

Idempotency – Means same config can run repeatedly safely , without creating duplicates.

Terraform destroy - Cloud engineers ALWAYS clean unused infrastructure to reduce cost, clutter, governance issues.

Sandbox Screenshots :

```
cloud_user_p_a7eb6b3a@cloudshell:~ (playground-s-11-536d1aa1)$ mkdir terraform-demo
cloud_user_p_a7eb6b3a@cloudshell:~ (playground-s-11-536d1aa1)$ cd terraform-demo
cloud_user_p_a7eb6b3a@cloudshell:~/terraform-demo (playground-s-11-536d1aa1)$ nano main.tf
cloud_user_p_a7eb6b3a@cloudshell:~/terraform-demo (playground-s-11-536d1aa1)$ terraform init
```

Initializing the backend...

Initializing provider plugins...

- Finding hashicorp/google versions matching "~> 5.0"...
- Installing hashicorp/google v5.45.2...
- Installed hashicorp/google v5.45.2 (signed by HashiCorp)

Terraform has created a lock file `.terraform.lock.hcl` to record the provider selections it made above. Include this file in your version control repository so that Terraform can guarantee to make the same selections by default when you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see any changes that are required for your infrastructure. All Terraform commands should now work.

If you ever set or change modules or backend configuration for Terraform, rerun this command to reinitialize your working directory. If you forget, other commands will detect it and remind you to do so if necessary.

```
cloud_user_p_a7eb6b3a@cloudshell:~/terraform-demo (playground-s-11-536d1aa1)$ terraform validate
Success! The configuration is valid.
```

```
cloud_user_p_a7eb6b3a@cloudshell:~/terraform-demo (playground-s-11-536d1aa1)$ terraform apply
```

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

```
# google_storage_bucket.demo_bucket will be created
+ resource "google_storage_bucket" "demo_bucket" {
+   effective_labels      = (known after apply)
+   force_destroy         = false
+   id                    = (known after apply)
+   location               = "US-CENTRAL1"
+   name                  = "maxx-terraform-demo-12345"
+   project                = (known after apply)
+   project_number        = (known after apply)
+   public_access_prevention = (known after apply)
+   rpo                   = (known after apply)
+   self_link              = (known after apply)
+   storage_class          = "STANDARD"
+   terraform_labels      = (known after apply)
+   uniform_bucket_level_access = (known after apply)
+   url                   = (known after apply)
}
```

Plan: 1 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?

Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

```

google_storage_bucket.demo_bucket: Creating...
google_storage_bucket.demo_bucket: Creation complete after 3s [id=maxx-terraform-demo-12345]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
cloud_user_p_a7eb6b3a@cloudshell:~/terraform-demo (playground-s-11-536d1aa1)$

```

Buckets Create Refresh Go to path Learn						
Filter Filter buckets ? ⚙ ☰						
☐ Name ↑	Created	Location type	Location	Default storage class ?	Last modified	Public access
☐ maxx-terraform-demo-12345	May 25, 2026, 10:48:26 PM	Region	us-central1	Standard	May 25, 2026, 10:48:26 PM	Value hidden ⋮

```

cloud_user_p_a7eb6b3a@cloudshell:~/terraform-demo (playground-s-11-536d1aa1)$ terraform destroy
google_storage_bucket.demo_bucket: Refreshing state... [id=maxx-terraform-demo-12345]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
- destroy

Terraform will perform the following actions:

# google_storage_bucket.demo_bucket will be destroyed
- resource "google_storage_bucket" "demo_bucket" {
  - default_event_based_hold = false -> null
  - effective_labels         = {} -> null
  - enable_object_retention = false -> null
  - force_destroy           = false -> null
  - id                      = "maxx-terraform-demo-12345" -> null
  - labels                  = {} -> null
  - location                = "US-CENTRAL1" -> null
  - name                    = "maxx-terraform-demo-12345" -> null
  - project                 = "playground-s-11-536d1aa1" -> null
  - project_number          = 440814540740 -> null
  - public_access_prevention = "inherited" -> null
  - requester_pays         = false -> null
  - self_link               = "https://www.googleapis.com/storage/v1/b/maxx-terraform-demo-12345" -> null
  - storage_class           = "STANDARD" -> null
  - terraform_labels       = {} -> null
  - uniform_bucket_level_access = false -> null
  - url                     = "gs://maxx-terraform-demo-12345" -> null

  - soft_delete_policy {
    - effective_time = "2026-05-25T17:18:26.183Z" -> null
    - retention_duration_seconds = 604800 -> null
  }
}

Plan: 0 to add, 0 to change, 1 to destroy.

```

```

Do you really want to destroy all resources?
Terraform will destroy all your managed infrastructure, as shown above.
There is no undo. Only 'yes' will be accepted to confirm.

Enter a value: yes

google_storage_bucket.demo_bucket: Destroying... [id=maxx-terraform-demo-12345]
google_storage_bucket.demo_bucket: Destruction complete after 2s

Destroy complete! Resources: 1 destroyed.
cloud_user_p_a7eb6b3a@cloudshell:~/terraform-demo (playground-s-11-536d1aa1)$

```

Task 12 — Kubernetes Workloads + Scaling + Self-Healing (Cloud-Native Core Skill)

Real-World Scenario - Company runs APIs , frontend apps , microservices , background workers

Questions become: “What if container crashes?” , “How do we scale traffic?” , “How do updates happen without downtime?” , “How are multiple replicas managed?”. This is where Kubernetes shines.

Solution :

- ❖ Go to Kubernetes and click on create cluster , choose specs and keep things default.
- ❖ Open cloud shell by going to top right corner , and run these :
mkdir k8s-demo → cd k8s-demo → nano deployment.yaml [create Kubernetes manifest file] , paste this:
apiVersion: apps/v1
kind: Deployment

```
metadata:  
  name: nginx-deployment
```

```
spec:  
  replicas: 2
```

```
selector:  
  matchLabels:  
    app: nginx
```

```
template:  
  metadata:  
    labels:  
      app: nginx
```

```
spec:  
  containers:  
    - name: nginx-container  
      image: nginx  
      ports:  
        - containerPort: 80
```

Save by pressing : Ctrl O + Enter + Ctrl X

gcloud container clusters get-credentials CLUSTER_NAME --region REGION [Connect kubectl to cluster , replace cluster name and region , with your configs you just created]

kubectl get nodes [Verify kubectl connection]

kubectl apply -f deployment.yaml [Apply Deployment]

kubectl get pods [Verify pods] – for live updates you can use this too {kubectl get pods -w} , press ctrl C to exit

kubectl get deployments [Verify deployment]

- ❖ Deployment is created , You did NOT manually: create containers, assign ports, restart processes. Kubernetes orchestrated everything. Now test by scaling the application (horizontal scaling) :
kubectl scale deployment nginx-deployment --replicas=4 [Scale application , from 2 to 4]
{For descaling , just decrease the count of replicas}
kubectl get pods [Verify scaling]

Note : After applying the deployment, Wait for few minutes since containers don't instantly appear. Kubernetes also has

self-healing feature, so if any pods gets deleted/terminated due to any reason, it will create a new pod automatically, since it prioritizes maintaining desired application state continuously {eg : desired 4 , actual 3 then it creates new immediately}

{In the below screenshots, I selected e2-micro configuration , so it keeps on failing whenever the 4th instance is being created due to limited resources, so to avoid this, select medium level configuration}

Concepts :

Kubernetes is not just running containers, It is container orchestration platform, Meaning manages containers, monitors them, scales them, heals them, updates them Automatically.

Pod = smallest deployable unit in Kubernetes , Usually contains: one container, networking, storage context.

Containers DO NOT directly run in Kubernetes.They run INSIDE: Pods

Real Life Analogy - Think: Container = application , Pod = protective wrapper/room for app

Deployment = controller managing Pods automatically. It ensures desired replica count, updates, recovery.

In real Kubernetes: you rarely create Pods directly , Usually: Deployments manage Pods for you

Desired State - You declare: "I want 3 replicas running." Kubernetes continuously works to maintain that state. This is declarative orchestration. Real Example , If one Pod crashes: Kubernetes automatically creates another. WITHOUT human intervention. This Is Called self-healing

Kubernetes resources are usually defined using: declarative YAML manifests

apiVersion - apps/v1 , Defines: which Kubernetes API version handles this resource

kind – Deployment , Means: resource type

metadata - Contains: name, labels, identifiers

replicas – eg: replicas: 2 Means: maintain 2 running Pod instances

selector - Defines: which Pods belong to Deployment

template - Blueprint for Pod creation.

image – nginx , Container image to run.

kubectl is Kubernetes command-line client , like: gcloud for GCP, docker CLI for Docker. Common Commands

Command	Meaning
kubectl apply	create/update resources
kubectl get pods	list pods
kubectl describe	detailed info
kubectl logs	container logs
kubectl delete	remove resources

Kubectl is a client tool , real life analogy - kubectl = TV remote , Kubernetes cluster = actual TV

gcloud container clusters get-credentials CLUSTER_NAME --region REGION - This command: downloads Kubernetes authentication + endpoint info into: kubeconfig. Without this: kubectl has no idea which cluster to talk to

Sandbox screenshots:

```
Welcome to Cloud Shell! Type "help" to get started, or type "gemini" to try prompting with Gemini CLI.
Your Cloud Platform project in this session is set to playground-s-11-15a67bfe.
Use `gcloud config set project [PROJECT_ID]` to change to a different project.
cloud_user_p_92c2871e@cloudshell:~ (playground-s-11-15a67bfe)$ mkdir k8s-demo
cloud_user_p_92c2871e@cloudshell:~ (playground-s-11-15a67bfe)$ cd k8s-demo
cloud_user_p_92c2871e@cloudshell:~/k8s-demo (playground-s-11-15a67bfe)$ nano deployment.yaml
```

```
cloud_user_p_92c2871e@cloudshell:~/k8s-demo (playground-s-11-15a67bfe)$ gcloud container clusters get-credentials cluster-1 --region us-central1-a
Fetching cluster endpoint and auth data.
kubeconfig entry generated for cluster-1.
cloud_user_p_92c2871e@cloudshell:~/k8s-demo (playground-s-11-15a67bfe)$ kubectl get nodes
NAME                                STATUS    ROLES    AGE      VERSION
gke-cluster-1-default-pool-46f86e66-012w Ready    <none>    3m28s    v1.35.3-gke.1389000
gke-cluster-1-default-pool-46f86e66-663h Ready    <none>    3m30s    v1.35.3-gke.1389000
gke-cluster-1-default-pool-46f86e66-g1z7 Ready    <none>    3m27s    v1.35.3-gke.1389000
cloud_user_p_92c2871e@cloudshell:~/k8s-demo (playground-s-11-15a67bfe)$ kubectl apply -f deployment.yaml
deployment.apps/nginx-deployment created
cloud_user_p_92c2871e@cloudshell:~/k8s-demo (playground-s-11-15a67bfe)$ kubectl get pods
NAME                                READY    STATUS              RESTARTS   AGE
nginx-deployment-c475b744d-27ctb    0/1      ContainerCreating    0          14s
nginx-deployment-c475b744d-cctsr    0/1      ContainerCreating    0          14s
```

```
cloud_user_p_92c2871e@cloudshell:~/k8s-demo (playground-s-11-15a67bfe)$ kubectl get pods -w
NAME                                READY    STATUS              RESTARTS   AGE
nginx-deployment-c475b744d-27ctb    0/1      ContainerCreating    0          6m
nginx-deployment-c475b744d-cctsr    1/1      Running              0          6m
nginx-deployment-c475b744d-27ctb    1/1      Running              0          6m13s
q
^Ccloud_user_p_92c2871e@cloudshell:~/k8s-demo (playground-s-11-15a67bfe)$ kubectl get deployments
NAME            READY   UP-TO-DATE   AVAILABLE   AGE
nginx-deployment 2/2     2            2           6m54s
cloud_user_p_92c2871e@cloudshell:~/k8s-demo (playground-s-11-15a67bfe)$ kubectl scale deployment nginx-deployment --replicas=4
deployment.apps/nginx-deployment scaled
```

```
cloud_user_p_92c2871e@cloudshell:~/k8s-demo (playground-s-11-15a67bfe)$ kubectl get pods -w
NAME                                READY    STATUS              RESTARTS   AGE
nginx-deployment-c475b744d-27ctb    1/1      Running              0          8m14s
nginx-deployment-c475b744d-cctsr    1/1      Running              0          8m14s
nginx-deployment-c475b744d-jms9t    0/1      Pending              0          41s
nginx-deployment-c475b744d-nfdjl    0/1      ContainerCreating    0          41s
nginx-deployment-c475b744d-nfdjl    1/1      Running              0          2m59s
nginx-deployment-c475b744d-jms9t    0/1      ContainerCreating    0          4m58s
^Ccloud_user_p_92c2871e@cloudshell:~/k8s-demo (playground-s-11-15a67bfe)$ kubectl get pods -w
NAME                                READY    STATUS              RESTARTS   AGE
nginx-deployment-c475b744d-27ctb    1/1      Running              0          19m
nginx-deployment-c475b744d-4t4b4    0/1      ContainerCreating    0          3s
nginx-deployment-c475b744d-cctsr    1/1      Running              0          19m
nginx-deployment-c475b744d-jms9t    0/1      Terminating        0          11m
nginx-deployment-c475b744d-nfdjl    1/1      Running              0          11m
nginx-deployment-c475b744d-jms9t    0/1      Terminating        0          12m
^Ccloud_user_p_92c2871e@cloudshell:~/k8s-demo (playground-s-11-15a67bfe)$ kubectl get deployment
NAME            READY   UP-TO-DATE   AVAILABLE   AGE
nginx-deployment 3/4     4            3           20m
```

```

^Ccloud_user_p_92c2871e@cloudshell:~/k8s-demo (playground-s-11-15a67bfe)$ kubectl get deployment
NAME                READY   UP-TO-DATE   AVAILABLE   AGE
nginx-deployment    3/4     4             3           20m
cloud_user_p_92c2871e@cloudshell:~/k8s-demo (playground-s-11-15a67bfe)$ kubectl get pods -w
NAME                READY   STATUS              RESTARTS   AGE
nginx-deployment-c475b744d-27ctb  1/1     Running             0          20m
nginx-deployment-c475b744d-4t4b4  1/1     Running             0          86s
nginx-deployment-c475b744d-cctsr  1/1     Running             0          20m
nginx-deployment-c475b744d-jms9t  0/1     Terminating       0          13m
nginx-deployment-c475b744d-nfdjl  1/1     Running             0          13m
nginx-deployment-c475b744d-jms9t  0/1     ContainerStatusUnknown 0          13m
nginx-deployment-c475b744d-jms9t  0/1     ContainerStatusUnknown 0          13m
nginx-deployment-c475b744d-jms9t  0/1     ContainerStatusUnknown 0          13m
nginx-deployment-c475b744d-4t4b4  1/1     Running             0          3m25s
nginx-deployment-c475b744d-cctsr  1/1     Running             0          22m
^Ccloud_user_p_92c2871e@cloudshell:~/k8s-demo (playground-s-11-15a67bfe)$ kubectl get pods
NAME                READY   STATUS    RESTARTS   AGE
nginx-deployment-c475b744d-27ctb  1/1     Running   0          23m
nginx-deployment-c475b744d-4t4b4  1/1     Running   0          4m6s
nginx-deployment-c475b744d-cctsr  1/1     Running   0          23m
nginx-deployment-c475b744d-nfdjl  1/1     Running   0          15m
cloud_user_p_92c2871e@cloudshell:~/k8s-demo (playground-s-11-15a67bfe)$ kubectl get deployment
NAME                READY   UP-TO-DATE   AVAILABLE   AGE
nginx-deployment    2/4     4             2           23m

```

```

^Ccloud_user_p_92c2871e@cloudshell:~/k8s-demo (playground-s-11-15a67bfe)$ kubectl scale deployment nginx-deployment --replicas=2
deployment.apps/nginx-deployment scaled
cloud_user_p_92c2871e@cloudshell:~/k8s-demo (playground-s-11-15a67bfe)$ kubectl get pods
NAME                READY   STATUS              RESTARTS   AGE
nginx-deployment-c475b744d-27ctb  1/1     Running             0          30m
nginx-deployment-c475b744d-4t4b4  1/1     Terminating       0          11m
nginx-deployment-c475b744d-7m7zb  0/1     Terminating       0          3m
nginx-deployment-c475b744d-cctsr  1/1     Terminating       0          30m
nginx-deployment-c475b744d-nfdjl  1/1     Running             0          23m
nginx-deployment-c475b744d-rfbk5  0/1     Terminating       0          3m
cloud_user_p_92c2871e@cloudshell:~/k8s-demo (playground-s-11-15a67bfe)$ kubectl delete deployment nginx-deployment
deployment.apps "nginx-deployment" deleted from default namespace
cloud_user_p_92c2871e@cloudshell:~/k8s-demo (playground-s-11-15a67bfe)$ kubectl get pods
NAME                READY   STATUS              RESTARTS   AGE
nginx-deployment-c475b744d-4t4b4  1/1     Terminating       0          11m
nginx-deployment-c475b744d-7m7zb  0/1     Terminating       0          3m22s
nginx-deployment-c475b744d-cctsr  1/1     Terminating       0          31m
nginx-deployment-c475b744d-rfbk5  0/1     Terminating       0          3m22s
cloud_user_p_92c2871e@cloudshell:~/k8s-demo (playground-s-11-15a67bfe)$ █

```

Task 13 — Kubernetes Services + Internal Networking + Load Balancing

Now you've learned: Pods, Deployments, Scaling, Self-healing. BUT there's a HUGE problem right now: Pods are temporary, They die, restart, get recreated, receive new IPs. So...“How do users consistently access applications?”

This is where: Kubernetes Services come in.

What you'll build : Create GKE Cluster → Deploy nginx application → Expose app using Kubernetes Service → Get public external IP → Access app in browser

Solution :

- Go to Kubernetes engine, click on cluster and create one. Select e2-small for config.
- Click connect to kubectl from the above, or just open cloud shell and run this : `gcloud container clusters get-credentials CLUSTERNAME --region REGION` [replace clustername and region with yours].

- Run these then in kubectl :

```
kubectl get nodes [Verify connection]
```

```
nano deployment.yaml [Create deployment], paste this inside it:
```

```
apiVersion: apps/v1
```

```
kind: Deployment
```

```
metadata:
```

```
  name: nginx-deployment
```

```
spec:
```

```
  replicas: 2
```

```
  selector:
```

```
    matchLabels:
```

```
      app: nginx
```

```
  template:
```

```
    metadata:
```

```
      labels:
```

```
        app: nginx
```

```
    spec:
```

```
      containers:
```

```
        - name: nginx-container
```

```
          image: nginx
```

```
          ports:
```

```
            - containerPort: 80
```

```
Save it : Ctrl O + Enter + Ctrl X
```

```
kubectl apply -f deployment.yaml [Apply Deployment]
```

```
kubectl get pods [Verify pods]
```

- Run : nano service.yaml [Create service YAML – expose it publicly], paste this inside it:

```
apiVersion: v1
```

```
kind: Service
```

```
metadata:
```

```
  name: nginx-service
```

```
spec:
  selector:
    app: nginx
```

```
ports:
  - protocol: TCP
    port: 80
    targetPort: 80
```

```
type: LoadBalancer
```

Save it : Ctrl O + Enter + Ctrl X

```
kubectl apply -f service.yaml [Apply Service]
```

```
kubectl get services [View Service]
```

- Wait for sometime till external IP appears , once it appears open it in new window , you should see the webpage
You just: deployed containers, exposed Kubernetes workloads, provisioned cloud load balancer, accessed distributed application publicly.
What's happening internally : User → Cloud Load Balancer → Kubernetes Service → Pods
- `kubectl delete pod POD_NAME` [Test self-healing, delete one pod , you'll see another one gets created immediately]
- Cleanup, First Delete service then deployment then cluster :
`kubectl delete service nginx-service`
`kubectl delete deployment nginx-deployment`
Delete cluster from : Kubernetes Engine → Clusters

Concepts :

Deployment manages: application lifecycle | Service manages: network access to Pods

Users should NEVER directly access Pods. Because Pods are: ephemeral , Instead: Services provide stable networking layer

Real Life Analogy - Think: Pods = restaurant workers changing shifts | Service = restaurant reception counter

Customers always contact: stable front desk , NOT individual workers.

Kubernetes Service - Service = stable network endpoint for accessing Pods. It: load balances traffic, tracks healthy Pods, provides stable IP/DNS. Even if Pods restart: Service endpoint remains stable

Understanding Service YAML - You just defined: stable network access layer for your application.

Understanding Each Part :

selector - app: nginx - This tells Service: which Pods belong to this Service based on labels.

labels: app: nginx - That's how Service discovers Pods.

port: 80 | targetPort: 80 - Means: Service listens on 80, forwards traffic to container port 80.

type: LoadBalancer - This tells GKE: create external cloud load balancer with public IP.

After applying service – what happens internally - Kubernetes: creates Service object , GKE then: provisions load balancer, allocates external IP, routes traffic to Pods.

Kubernetes integrates with: actual cloud infrastructure APIs to create networking resources automatically.

Deleting one pod – what happens - Service still works, Deployment recreates Pod, traffic routes to healthy Pods.
WITHOUT downtime. You just experienced: resilient distributed application networking

TYPES OF KUBERNETES SERVICES :

1. ClusterIP - Default. - internal-only communication - Used for: backend APIs, database communication, microservices.
2. NodePort - Exposes application through: node IP + port. Mostly for: testing, development.
3. LoadBalancer - Cloud provider creates: external managed load balancer | Production common.
4. ExternalName - Maps Service to: external DNS name

Services provide: service discovery inside cluster.

Example - Frontend Pod can call: `http://backend-service`, instead of Pod IPs.

Cloud-Native Benefit - Applications become: loosely coupled and resilient.

Sandbox Screenshots :

```
Welcome to Cloud Shell! Type "help" to get started, or type "gemini" to try prompting with Gemini CLI.
Your Cloud Platform project in this session is set to playground-s-11-25b55ae6.
Use 'gcloud config set project [PROJECT_ID]' to change to a different project.
gcloud container clusters get-credentials nginx-cluster --zone us-central1-a --project playground-s-11-25b55ae6cloud_user_p_c2685be2@cloudshell:~ (playground-s-11-25b55ae6)$ gcloud container
clusters get-credentials nginx-cluster --zone us-central1-a --project playground-s-11-25b55ae6
Fetching cluster endpoint and auth data.
kubeconfig entry generated for nginx-cluster.
cloud_user_p_c2685be2@cloudshell:~ (playground-s-11-25b55ae6)$ kubectl get nodes
NAME                                STATUS    ROLES    AGE   VERSION
gke-nginx-cluster-default-pool-0b53eaaa-68b1    Ready    <none>    90s   v1.35.3-gke.1389000
gke-nginx-cluster-default-pool-0b53eaaa-7533    Ready    <none>    90s   v1.35.3-gke.1389000
gke-nginx-cluster-default-pool-0b53eaaa-q396    Ready    <none>    94s   v1.35.3-gke.1389000
cloud_user_p_c2685be2@cloudshell:~ (playground-s-11-25b55ae6)$ nano deployment.yaml
cloud_user_p_c2685be2@cloudshell:~ (playground-s-11-25b55ae6)$ kubectl apply -f deployment.yaml
deployment.apps/nginx-deployment created
cloud_user_p_c2685be2@cloudshell:~ (playground-s-11-25b55ae6)$ kubectl get pods
NAME                                READY    STATUS    RESTARTS   AGE
nginx-deployment-c475b744d-bngww    1/1      Running    0           8s
nginx-deployment-c475b744d-z54hs    0/1      ContainerCreating    0           8s
cloud_user_p_c2685be2@cloudshell:~ (playground-s-11-25b55ae6)$ kubectl get pods -w
NAME                                READY    STATUS    RESTARTS   AGE
nginx-deployment-c475b744d-bngww    1/1      Running    0          16s
nginx-deployment-c475b744d-z54hs    1/1      Running    0          16s
^Ccloud_user_p_c2685be2@cloudshell:~ (playground-s-11-25b55ae6)$ nano service.yaml
cloud_user_p_c2685be2@cloudshell:~ (playground-s-11-25b55ae6)$ kubectl apply -f service.yaml
service/nginx-service created
cloud_user_p_c2685be2@cloudshell:~ (playground-s-11-25b55ae6)$ kubectl get services
NAME      TYPE        CLUSTER-IP    EXTERNAL-IP    PORT(S)    AGE
kubernetes  ClusterIP   34.118.224.1  <none>         443/TCP    4m40s
nginx-service  LoadBalancer  34.118.229.177 <pending>     80:30995/TCP 11s
```

```
cloud_user_p_c2685be2@cloudshell:~ (playground-s-11-25b55ae6)$ kubectl get services
NAME            TYPE        CLUSTER-IP    EXTERNAL-IP    PORT(S)    AGE
kubernetes      ClusterIP   34.118.224.1  <none>         443/TCP    5m38s
nginx-service   LoadBalancer  34.118.229.177 104.197.56.143 80:30995/TCP 69s
```

Welcome to nginx!

If you see this page, nginx is successfully installed and working. Further configuration is required for the web server, reverse proxy, API gateway, load balancer, content cache, or other features.

For online documentation and support please refer to nginx.org.
 To engage with the community please visit community.nginx.org.
 For enterprise grade support, professional services, additional security features and capabilities please refer to f5.com/nginx.

Thank you for using nginx.

```
cloud_user_p_c2685be2@cloudshell:~ (playground-s-11-25b55ae6)$ kubectl delete pod nginx-deployment-c475b744d-bngww
pod "nginx-deployment-c475b744d-bngww" deleted from default namespace
cloud_user_p_c2685be2@cloudshell:~ (playground-s-11-25b55ae6)$ kubectl get pods -w
NAME                                READY   STATUS    RESTARTS   AGE
nginx-deployment-c475b744d-t8g8j    1/1     Running   0           7s
nginx-deployment-c475b744d-z54hs    1/1     Running   0          5m11s
```

```
^Ccloud_user_p_c2685be2@cloudshell:~ (playground-s-11-25b55ae6)$ kubectl delete service nginx-service
service "nginx-service" deleted from default namespace
^Ccloud_user_p_c2685be2@cloudshell:~ (playground-s-11-25b55ae6)$ kubectl delete deployment nginx-deployment
deployment.apps "nginx-deployment" deleted from default namespace
cloud_user_p_c2685be2@cloudshell:~ (playground-s-11-25b55ae6)$
```

Task 14 — Kubernetes ConfigMaps + Secrets + Environment Variables (REAL Production Pattern)

This is one of the MOST used Kubernetes concepts in real enterprise projects. Because applications need: database URLs, API endpoints, feature flags, credentials, tokens, passwords, environment-specific configs.

And hardcoding them inside application code is: VERY bad practice

REAL-WORLD SCENARIO - Imagine company app needs: DB_HOST=prod-db.company.internal. If this is hardcoded: changing environment becomes difficult, deployments become messy, security risks increase.

So Kubernetes separates: application code FROM configuration

Solution :

- ❖ Go to Kubernetes engine , Click on create cluster , Select e2-small config rest everything at default.
- ❖ gcloud container clusters get-credentials config-cluster --region REGION [Connect to kubectl]
- ❖ Run these :

```
kubectl get nodes [Verify connection]
kubectl create configmap app-config \ [Create Config map]
--from-literal=APP_MODE=production \
--from-literal=APP_COLOR=blue
kubectl get configmaps [Verify config map]
kubectl describe configmap app-config [View details]
kubectl create secret generic app-secret \ [Create Secret]
--from-literal=DB_PASSWORD=mysecurepassword
kubectl get secrets [Verify secrets]
kubectl describe secret app-secret [View secrets]
nano deployment.yaml [create deployment yaml] , paste this inside :
apiVersion: apps/v1
kind: Deployment
```

```
metadata:
  name: nginx-config-demo
```

```
spec:
  replicas: 1
```

```
selector:
  matchLabels:
    app: nginx-config
```

```
template:
  metadata:
    labels:
      app: nginx-config
```

```
spec:
  containers:
    - name: nginx
      image: nginx
```

```
env:
- name: APP_MODE
  valueFrom:
    configMapKeyRef:
      name: app-config
      key: APP_MODE

- name: DB_PASSWORD
  valueFrom:
    secretKeyRef:
      name: app-secret
      key: DB_PASSWORD
```

Save file : Ctrl O + Enter + Ctrl X

- ❖ `kubectl apply -f deployment.yaml` [Apply Deployment]
- `kubectl get pods` [Verify pods]
- ❖ `kubectl exec -it POD_NAME -- /bin/bash` [Access pod shell – rename pod name with yours]
- Inside container , run : `printenv`
- ❖ You just: injected ConfigMap values, injected Secrets, externalized application configuration, used runtime environment variables.
- ❖ Cleanup : Run :
 - `kubectl delete deployment nginx-config-demo`
 - `kubectl delete configmap app-config`
 - `kubectl delete secret app-secret`
 - and delete cluster from Kubernetes dashboard.

Concept :

ConfigMap = Kubernetes object storing non-sensitive configuration , Examples: app mode, API URL, feature flags, port configs

Secret = Kubernetes object storing sensitive data. Examples: passwords, tokens, API keys, credentials. Secrets are base64 encoded

When you view the secrets, the password gets hidden. Kubernetes attempts safer sensitive data handling

When you saved the deployment file, You just defined: externalized configuration injection. Container image: remains unchanged, But runtime configuration changes dynamically.

Exec - allows: running commands inside container.

`printenv` - Linux command: prints environment variables

Important cloud native principle - Applications SHOULD read configuration from: environment variables , NOT hardcoded values. This matters because environments differ, secrets rotate, deployments change frequently.

Important commands :

Command	Meaning
<code>kubectl create configmap</code>	create configuration
<code>kubectl create secret</code>	create secret storage
<code>kubectl describe</code>	inspect resource

Command	Meaning
kubectl exec	run inside container
printenv	view env variables

Sandbox Screenshots :

```
Welcome to Cloud Shell! Type "help" to get started, or type "gemini" to try prompting with Gemini CLI.
Your Cloud Platform project in this session is set to playground-s-11-1a8e4e39.
Use 'gcloud config set project [PROJECT_ID]' to change to a different project.
gcloud container clusters get-credentials config-cluster --zone us-central1-a --project playground-s-11-1a8e4e39cloud_user_p_20cf58a0@cloudshell:~ (playground-s-11-1a8e4e39)$ gcloud container
clusters get-credentials config-cluster --zone us-central1-a --project playground-s-11-1a8e4e39
Fetching cluster endpoint and auth data.
kubeconfig entry generated for config-cluster.
cloud_user_p_20cf58a0@cloudshell:~ (playground-s-11-1a8e4e39)$ kubectl get nodes
NAME                                STATUS  ROLES  AGE   VERSION
gke-config-cluster-default-pool-6c52afbe-g5xl  Ready  <none>  3m57s  v1.35.3-gke.1389000
gke-config-cluster-default-pool-6c52afbe-vxwf  Ready  <none>  3m56s  v1.35.3-gke.1389000
gke-config-cluster-default-pool-6c52afbe-xmjc  Ready  <none>  3m55s  v1.35.3-gke.1389000
cloud_user_p_20cf58a0@cloudshell:~ (playground-s-11-1a8e4e39)$ kubectl create configmap app-config \
--from-literal=APP_MODE=production \
--from-literal=APP_COLOR=blue
configmap/app-config created
cloud_user_p_20cf58a0@cloudshell:~ (playground-s-11-1a8e4e39)$ kubectl get configmaps
NAME          DATA  AGE
app-config    2      19s
kube-root-ca.crt  1      5m51s
```

```
cloud_user_p_20cf58a0@cloudshell:~ (playground-s-11-1a8e4e39)$ kubectl describe configmap app-config
Name:         app-config
Namespace:    default
Labels:       <none>
Annotations:  <none>

Data
====
APP_COLOR:
----
blue

APP_MODE:
----
production

BinaryData
=====

Events:  <none>
cloud_user_p_20cf58a0@cloudshell:~ (playground-s-11-1a8e4e39)$ kubectl create secret generic app-secret \
--from-literal=DB_PASSWORD=mysecurepassword
secret/app-secret created
cloud_user_p_20cf58a0@cloudshell:~ (playground-s-11-1a8e4e39)$ kubectl get secrets
NAME          TYPE      DATA  AGE
app-secret    Opaque    1      10s
```

```
cloud_user_p_20cf58a0@cloudshell:~ (playground-s-11-1a8e4e39)$ kubectl describe secret app-secret
Name:          app-secret
Namespace:     default
Labels:        <none>
Annotations:   <none>

Type: Opaque

Data
====
DB_PASSWORD:  16 bytes
cloud_user_p_20cf58a0@cloudshell:~ (playground-s-11-1a8e4e39)$ nano deployment.yaml
cloud_user_p_20cf58a0@cloudshell:~ (playground-s-11-1a8e4e39)$ kubectl apply -f deployment.yaml
deployment.apps/nginx-config-demo created
cloud_user_p_20cf58a0@cloudshell:~ (playground-s-11-1a8e4e39)$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
nginx-config-demo-777869497c-9qgmk  1/1     Running   0           8s
```

```
cloud_user_p_20cf58a0@cloudshell:~ (playground-s-11-1a8e4e39)$ kubectl exec -it nginx-config-demo-777869497c-9qgmk -- /bin/bash
root@nginx-config-demo-777869497c-9qgmk:/# printenv
DB_PASSWORD=mysecurepassword
KUBERNETES_SERVICE_PORT_HTTPS=443
KUBERNETES_SERVICE_PORT=443
HOSTNAME=nginx-config-demo-777869497c-9qgmk
PWD=/
ACME_VERSION=0.4.1
PKG_RELEASE=1~trixie
HOME=/root
KUBERNETES_PORT_443_TCP=tcp://34.118.224.1:443
DYNPKG_RELEASE=1~trixie
NJS_VERSION=0.9.9
TERM=xterm
SHLVL=1
KUBERNETES_PORT_443_TCP_PROTO=tcp
KUBERNETES_PORT_443_TCP_ADDR=34.118.224.1
APP_MODE=production
KUBERNETES_SERVICE_HOST=34.118.224.1
KUBERNETES_PORT=tcp://34.118.224.1:443
KUBERNETES_PORT_443_TCP_PORT=443
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
NGINX_VERSION=1.31.1
NJS_RELEASE=1~trixie
=/usr/bin/printenv
```

```
cloud_user_p_20cf58a0@cloudshell:~ (playground-s-11-1a8e4e39)$ kubectl delete deployment nginx-config-demo
deployment.apps "nginx-config-demo" deleted from default namespace
cloud_user_p_20cf58a0@cloudshell:~ (playground-s-11-1a8e4e39)$ kubectl delete configmap app-config
configmap "app-config" deleted from default namespace
cloud_user_p_20cf58a0@cloudshell:~ (playground-s-11-1a8e4e39)$ kubectl delete secret app-secret
secret "app-secret" deleted from default namespace
cloud_user_p_20cf58a0@cloudshell:~ (playground-s-11-1a8e4e39)$
```

Task 15 — Kubernetes Persistent Storage + Volumes + Stateful Data

What happens to data when containers die? Because containers are: temporary, replaceable, ephemeral.

So if application stores files INSIDE container: data disappears when Pod restarts. This is HUGE in real systems.

REAL-WORLD SCENARIO - Imagine: database container, uploads folder, logs, user files. If Pod crashes: data must survive, Otherwise: databases corrupt, users lose files, applications fail.

Solution :

- Go to Kubernetes Engine, Create cluster with e2-small configuration.
- Connect kubectl : gcloud container clusters get-credentials storage-cluster --region {your-region-here}
- kubectl get nodes – [Verify]
- nano pvc.yaml – [Create PVC yaml] , paste this :
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
 name: nginx-pvc
spec:
 accessModes:
 - ReadWriteOnce
 resources:
 requests:
 storage: 1Gi
Save it : Ctrl O + Enter + Ctrl X
- kubectl apply -f pvc.yaml – [Apply PVC]
- kubectl get pvc – [Verify PVC]
- nano deployment.yaml - [Create deployment using storage] , paste this :
apiVersion: apps/v1
kind: Deployment
metadata:
 name: nginx-storage-demo
spec:
 replicas: 1
 selector:
 matchLabels:
 app: nginx-storage
 template:
 metadata:
 labels:
 app: nginx-storage
 spec:
 containers:
 - name: nginx
 image: nginx
 volumeMounts:
 - mountPath: "/usr/share/nginx/html/data"
 name: storage-volume
 volumes:
 - name: storage-volume

persistentVolumeClaim:

claimName: nginx-pvc

Save it : Ctrl O + Enter + Ctrl X

- kubectl apply -f deployment.yaml – [Apply Deployment]
- kubectl get pods – [Verify pod]
- kubectl exec -it POD_NAME -- /bin/bash – [Access Container , replace PODNAME with yours]
- echo "Persistent Kubernetes Storage" > /usr/share/nginx/html/data/test.txt – [Create file inside persistent storage]
- cat /usr/share/nginx/html/data/test.txt – [Verify file]
- exit and delete the pod [Test persistence]
- kubectl exec -it NEW_POD_NAME -- /bin/bash [Verify data still exists] , run this too after accessing new pod :
cat /usr/share/nginx/html/data/test.txt
- You should still see : Persistent Kubernetes Storage

[Note : Persistent Volume Claims (PVCs) using the WaitForFirstConsumer binding mode remain in a Pending state until a Pod that consumes the claim is scheduled. This allows Kubernetes to provision storage in the appropriate zone or node location before binding the volume. In restricted sandbox environments, dynamic storage provisioning may be limited, causing PVCs to remain pending even when cluster resources are healthy.

Final status : Concept Understanding : (Completed) → Cluster Validation : (Completed) → PVC Troubleshooting : (Completed) → Storage Provisioning : ⚠ Not completed due to sandbox limitation]

Concepts :

State = data that must persist over time , Examples: database records, uploaded images, user sessions, logs.

Volume = storage mounted into container , Like: attaching drive/folder to application

Container filesystem alone is: temporary , Volumes provide: persistent or shared storage

PersistentVolume (PV) = actual storage resource in cluster , Could map to: GCP Persistent Disk , NFS , SSD , cloud storage backend

PersistentVolumeClaim (PVC) = request for storage by application. Think: PV = actual hard drive , PVC = application requesting disk

Kubernetes abstracts: storage consumption from storage implementation

PVC YAML - Requests: 1Gi persistent storage from Kubernetes storage system. You did NOT manually create disk.

Kubernetes dynamically provisions storage. Kubernetes automatically provisioned cloud storage resource behind the scenes.

Deployment yaml - This mounts persistent storage into container path

After creating and verifying file inside container , You just wrote data into persistent Kubernetes storage

After deleting pod , what happens - Pod died, container replaced, BUT storage persisted. REAL stateful workload behavior.

STATELESS vs STATEFUL -

Stateless Apps , Examples: frontend , APIs , microservices. Can restart safely without local data.

Stateful Apps , Examples: databases , file systems , message queues . Require: persistent durable storage.

Task 16 — Kubernetes Ingress + URL Routing + Single Entry Point

Up until now, you've exposed applications using: Service Type = LoadBalancer , which gives:

App A → External IP #1 , App B → External IP #2 , App C → External IP #3. That becomes expensive and messy in production.

Real-World Problem - Imagine a company has: Frontend | Backend API | Admin Portal , Without Ingress:

34.12.1.10 , 34.12.1.11 , 34.12.1.12 - Three separate public endpoints. Not ideal.

Real-World Solution - Use a single entry point: company.com , Then route traffic:

company.com → Frontend | company.com/api → Backend | company.com/admin → Admin Portal. This is exactly what Kubernetes Ingress does.

Solution :

- Go to Kubernetes engine , create cluster – e2-small. Connect to kubectl. Verify it by : kubectl get nodes.
- Create frontend-deployment [kubectl create deployment frontend --image=nginx]
- Create backend-deployment [kubectl create deployment backend --image=nginx]
Verify both with : kubectl get deployments
- Expose frontend internally [kubectl expose deployment frontend --port=80 --target-port=80]
- Expose backend-internally [kubectl expose deployment backend --port=80 --target-port=80]
Verify services : kubectl get services
- Create ingress yaml - nano ingress.yaml , paste this inside :
apiVersion: networking.k8s.io/v1
kind: Ingress

metadata:

name: company-ingress

spec:

defaultBackend:

service:

name: frontend

port:

number: 80

Save it : Ctrl O + Enter + Ctrl X

- Apply Ingress - kubectl apply -f ingress.yaml
- Check Ingress - kubectl get ingress
- Observe resources - kubectl get ingress

Concepts :

ClusterIP means Accessible only inside cluster, No public IP. This is the most common service type in production.

Kubernetes Ingress is an API object that manages external access to services within a cluster, typically routing HTTP and HTTPS traffic. It acts as a smart reverse proxy and entry point, allowing multiple services to be exposed via a single IP address using rules based on hostnames or URL paths. Ingress in GKE often creates: Load Balancer, Backend Services, Health Checks, Forwarding Rules.

After applying ingress , when you see the address , That means GKE created: External Load Balancer, Health Checks, Backend Services automatically.

Reverse Proxy - Ingress acts as a reverse proxy. User requests: company.com, Ingress decides where traffic goes.

Sandbox Screenshots :

Welcome to nginx!

If you see this page, nginx is successfully installed and working. Further configuration is required for the web server, reverse proxy, API gateway, load balancer, content cache, or other features.

For online documentation and support please refer to nginx.org.

To engage with the community please visit community.nginx.org.

For enterprise grade support, professional services, additional security features and capabilities please refer to f5.com/nginx.

Thank you for using nginx.

```
Welcome to Cloud Shell! Type "help" to get started, or type "gemini" to try prompting with Gemini CLI.
Your Cloud Platform project in this session is set to playground-s-11-8ce39c8f.
Use 'gcloud config set project [PROJECT_ID]' to change to a different project.
gcloud container clusters get-credentials ingress-cluster --zone us-central1-a --project playground-s-11-8ce39c8fcloud_user_p_b9ab18d8@cloudshell:~ (playground-s-11-8ce39c8f) $ gcloud container clusters get-credentials ingress-cluster --zone us-central1-a --project playground-s-11-8ce39c8f
Fetching cluster endpoint and auth data.
kubeconfig entry generated for ingress-cluster.
cloud_user_p_b9ab18d8@cloudshell:~ (playground-s-11-8ce39c8f) $ kubectl get nodes
NAME                                STATUS    ROLES    AGE    VERSION
gke-ingress-cluster-default-pool-1df1d543-pcyp  Ready    <none>    80s    v1.35.3-gke.1389002
cloud_user_p_b9ab18d8@cloudshell:~ (playground-s-11-8ce39c8f) $ kubectl create deployment frontend --image=nginx
deployment.apps/frontend created
cloud_user_p_b9ab18d8@cloudshell:~ (playground-s-11-8ce39c8f) $ kubectl get deployment
NAME    READY   UP-TO-DATE   AVAILABLE   AGE
frontend 0/1      1            0           7s
cloud_user_p_b9ab18d8@cloudshell:~ (playground-s-11-8ce39c8f) $ kubectl create deployment backend --image=nginx
deployment.apps/backend created
cloud_user_p_b9ab18d8@cloudshell:~ (playground-s-11-8ce39c8f) $ kubectl get deployment
NAME    READY   UP-TO-DATE   AVAILABLE   AGE
backend 1/1      1            1           18s
frontend 1/1      1            1           52s
cloud_user_p_b9ab18d8@cloudshell:~ (playground-s-11-8ce39c8f) $ kubectl expose deployment frontend \
--port=80 \
--target-port=80
service/frontend exposed
cloud_user_p_b9ab18d8@cloudshell:~ (playground-s-11-8ce39c8f) $ kubectl expose deployment backend \
--port=80 \
--target-port=80
service/backend exposed
```

```
cloud_user_p_b9ab18d8@cloudshell:~ (playground-s-11-8ce39c8f) $ kubectl get services
NAME            TYPE        CLUSTER-IP      EXTERNAL-IP      PORT(S)        AGE
backend         ClusterIP   34.118.231.10   <none>           80/TCP         7s
frontend        ClusterIP   34.118.235.189  <none>           80/TCP         18s
kubernetes      ClusterIP   34.118.224.1    <none>           443/TCP        4m10s
cloud_user_p_b9ab18d8@cloudshell:~ (playground-s-11-8ce39c8f) $ nano ingress.yaml
cloud_user_p_b9ab18d8@cloudshell:~ (playground-s-11-8ce39c8f) $ kubectl apply -f ingress.yaml
ingress.networking.k8s.io/company-ingress created
cloud_user_p_b9ab18d8@cloudshell:~ (playground-s-11-8ce39c8f) $ kubectl get ingress
NAME            CLASS    HOSTS    ADDRESS    PORTS    AGE
company-ingress <none>   *        80         18s
cloud_user_p_b9ab18d8@cloudshell:~ (playground-s-11-8ce39c8f) $ kubectl get ingress
NAME            CLASS    HOSTS    ADDRESS    PORTS    AGE
company-ingress <none>   *        8.233.218.200  80       2m23s
```

Task 17 — CI/CD for Kubernetes (From Scratch)

Up to now, you've manually run commands like: `kubectl apply -f deployment.yaml`.

But in real companies: Developer pushes code → Git Repository → CI/CD Pipeline → Build Container → Deploy to Kubernetes. Nobody manually SSHs into servers and deploys things all day.

Goal - Understand how code automatically reaches Kubernetes.

Real-Life Scenario - Suppose you update: Welcome to Version 2, Instead of: Build image manually, Push image manually, Update deployment manually. the pipeline does it automatically.

Solution :

- Go to Kubernetes, create cluster with e2-small config, connect to kubectl and verify it.
- Deploy initial version : `kubectl create deployment webapp --image=nginx`
verify it : `kubectl get deployments` , view current deployment : `kubectl describe deployment webapp`
- Simulate pipeline deployment : `kubectl set image deployment/webapp nginx=nginx:latest`
- Check rollout status : `kubectl rollout status deployment/webapp`
- View rollout history : `kubectl rollout history deployment/webapp`
- Simulate a bad release : `kubectl set image deployment/webapp nginx=nginx:1.14`
Verify : `kubectl rollout status deployment/webapp`
Check history again : `kubectl rollout history deployment/webapp`

Concepts :

Kubernetes does NOT kill all Pods immediately. It performs: Rolling Update, meaning:

Old Pod → New Pod Created → Traffic Shifted → Old Pod Removed. Users experience little or no downtime.

Rollout history - When a release breaks production: Version 5 (breaks), you can rollback to: Version 4, without rebuilding everything.

Sandbox Screenshots :

```
Welcome to Cloud Shell! Type "help" to get started, or type "gemini" to try prompting with Gemini CLI.
Your Cloud Platform project in this session is set to playground-s-11-3a91defc.
Use 'gcloud config set project [PROJECT_ID]' to change to a different project.
gcloud container clusters get-credentials cicd-cluster --zone us-central1-a --project playground-s-11-3a91defc cloud_user_p_c5562086@cloudshell:~ (playground-s-11-3a91defc)$ gcloud container clusters get-credentials cicd-cluster --zone us-central1-a --project playground-s-11-3a91defc
Fetching cluster endpoint and auth data.
kubeconfig entry generated for cicd-cluster.
cloud_user_p_c5562086@cloudshell:~ (playground-s-11-3a91defc)$ kubectl get nodes
NAME                                STATUS    ROLES    AGE   VERSION
gke-cicd-cluster-default-pool-067173d3-ln4s   Ready    <none>   66s   v1.35.3-gke.1389002
gke-cicd-cluster-default-pool-067173d3-j15f   Ready    <none>   66s   v1.35.3-gke.1389002
gke-cicd-cluster-default-pool-067173d3-wk76   Ready    <none>   65s   v1.35.3-gke.1389002
cloud_user_p_c5562086@cloudshell:~ (playground-s-11-3a91defc)$ kubectl create deployment webapp --image=nginx
deployment.apps/webapp created
cloud_user_p_c5562086@cloudshell:~ (playground-s-11-3a91defc)$ kubectl get deployments
NAME    READY   UP-TO-DATE   AVAILABLE   AGE
webapp  1/1     1            1           14s
```



```
cloud_user_p_c5562086@cloudshell:~ (playground-s-11-3a91defc)$ kubectl describe deployment webapp
Name: webapp
Namespace: default
CreationTimestamp: Tue, 02 Jun 2026 12:06:50 +0000
Labels: app=webapp
Annotations: deployment.kubernetes.io/revision: 1
Selector: app=webapp
Replicas: 1 desired | 1 updated | 1 total | 1 available | 0 unavailable
StrategyType: RollingUpdate
MinReadySeconds: 0
RollingUpdateStrategy: 25% max unavailable, 25% max surge
Pod Template:
  Labels: app=webapp
  Containers:
    nginx:
      Image: nginx
      Port: <none>
      Host Port: <none>
      Environment: <none>
      Mounts: <none>
      Volumes: <none>
      Node-Selectors: <none>
      Tolerations: <none>
Conditions:
  Type           Status  Reason
  ----           -
  Available      True    MinimumReplicasAvailable
  Progressing    True    NewReplicaSetAvailable
OldReplicaSets: <none>
NewReplicaSet: webapp-645b5b46cd (1/1 replicas created)
Events:
  Type    Reason              Age   From                  Message
  ----    -
  Normal  ScalingReplicaSet   21s   deployment-controller  Scaled up replica set webapp-645b5b46cd from 0 to 1
cloud_user_p_c5562086@cloudshell:~ (playground-s-11-3a91defc)$ kubectl set image deployment/webapp nginx=nginx:latest
deployment.apps/webapp image updated
```

```
cloud_user_p_c5562086@cloudshell:~ (playground-s-11-3a91defc)$ kubectl rollout status deployment/webapp
deployment "webapp" successfully rolled out
cloud_user_p_c5562086@cloudshell:~ (playground-s-11-3a91defc)$ kubectl rollout history deployment/webapp
deployment.apps/webapp
REVISION  CHANGE-CAUSE
1         <none>
2         <none>

cloud_user_p_c5562086@cloudshell:~ (playground-s-11-3a91defc)$ kubectl set image deployment/webapp nginx=nginx:1.14
deployment.apps/webapp image updated
cloud_user_p_c5562086@cloudshell:~ (playground-s-11-3a91defc)$ kubectl rollout status deployment/webapp
deployment "webapp" successfully rolled out
cloud_user_p_c5562086@cloudshell:~ (playground-s-11-3a91defc)$ kubectl rollout history deployment/webapp
deployment.apps/webapp
REVISION  CHANGE-CAUSE
1         <none>
2         <none>
3         <none>
```

```
cloud_user_p_c5562086@cloudshell:~ (playground-s-11-3a91defc)$ kubectl describe deployment webapp
Name: webapp
Namespace: default
CreationTimestamp: Tue, 02 Jun 2026 12:06:50 +0000
Labels: app=webapp
Annotations: deployment.kubernetes.io/revision: 3
Selector: app=webapp
Replicas: 1 desired | 1 updated | 1 total | 1 available | 0 unavailable
StrategyType: RollingUpdate
MinReadySeconds: 0
RollingUpdateStrategy: 25% max unavailable, 25% max surge
Pod Template:
  Labels: app=webapp
  Containers:
    nginx:
      Image: nginx:1.14
      Port: <none>
      Host Port: <none>
      Environment: <none>
      Mounts: <none>
      Volumes: <none>
      Node-Selectors: <none>
      Tolerations: <none>
Conditions:
  Type           Status  Reason
  ----           -
  Available      True    MinimumReplicasAvailable
  Progressing    True    NewReplicaSetAvailable
OldReplicaSets: webapp-645b5b46cd (0/0 replicas created), webapp-67dcbfb8f (0/0 replicas created)
NewReplicaSet: webapp-64ccd67588 (1/1 replicas created)
Events:
  Type    Reason              Age   From                  Message
  ----    -
  Normal  ScalingReplicaSet   7m30s deployment-controller  Scaled up replica set webapp-645b5b46cd from 0 to 1
  Normal  ScalingReplicaSet   6m37s deployment-controller  Scaled up replica set webapp-67dcbfb8f from 0 to 1
  Normal  ScalingReplicaSet   6m29s deployment-controller  Scaled down replica set webapp-645b5b46cd from 1 to 0
  Normal  ScalingReplicaSet   5m4s  deployment-controller  Scaled up replica set webapp-64ccd67588 from 0 to 1
  Normal  ScalingReplicaSet   4m58s deployment-controller  Scaled down replica set webapp-67dcbfb8f from 1 to 0
```

Task 18 — Kubernetes Rolling Updates & Rollbacks (From Scratch)

Deployments don't always go like this: Version 1 → Version 2 → Everyone Happy.

Sometimes they go like: Version 1 → Version 2 → Production Breaks

And then your team lead says: "Rollback NOW." This task teaches exactly that.

Real-World Scenario - Imagine: E-commerce Website, Version 1 works perfectly. Developer releases Version 2.

Suddenly: Login broken, Checkout failing, Users complaining

You need to revert immediately. That's where Rollbacks save the day.

Solution :

- Go to Kubernetes, create cluster e2-small config, connect to kubectl and verify.
- Deploy version 1: kubectl create deployment webapp --image=nginx:1.25
Verify: kubectl get deployments | Check Pods: kubectl get pods
- View current Image: kubectl describe deployment webapp
- Release version 2: kubectl set image deployment/webapp nginx=nginx:1.26
Watch Rollout: kubectl rollout status deployment/webapp
View Revision history: kubectl rollout history deployment/webapp
- Simulate bad release: kubectl rollout undo deployment/webapp
Verify rollback: kubectl rollout status deployment/webapp , Then: kubectl describe deployment webapp
- View history again: kubectl rollout history deployment/webapp

Concepts :

Architecture - Version 1 → Deploy Version 2 → Issue Found → Rollback → Back to Version 1

Deployment Revision - Every time you change: image, replicas, pod template | Kubernetes creates a new revision.

Think: Revision 1 = Original version | Revision 2 = Updated version | Revision 3 = Another update

Like save points in a game.

Notice revisions continue growing. Kubernetes keeps deployment history for recovery purposes.

Sandbox Screenshots :

```
cloud_user_p_7290e748@cloudshell:~ (playground-s-11-5e9e7aff) $ gcloud container clusters get-credentials rollback-cluster --region us-central1-a
Fetching cluster endpoint and auth data.
kubeconfig entry generated for rollback-cluster.
cloud_user_p_7290e748@cloudshell:~ (playground-s-11-5e9e7aff) $ kubectl get nodes
NAME                                STATUS    ROLES    AGE    VERSION
gke-rollback-cluster-default-pool-3bfa837d-dvv6    Ready    <none>    6m10s    v1.35.3-gke.1389002
gke-rollback-cluster-default-pool-3bfa837d-fm72    Ready    <none>    6m10s    v1.35.3-gke.1389002
gke-rollback-cluster-default-pool-3bfa837d-p9nx    Ready    <none>    6m11s    v1.35.3-gke.1389002
cloud_user_p_7290e748@cloudshell:~ (playground-s-11-5e9e7aff) $ kubectl create deployment webapp --image=nginx:1.25
deployment.apps/webapp created
cloud_user_p_7290e748@cloudshell:~ (playground-s-11-5e9e7aff) $ kubectl get deployments
NAME    READY    UP-TO-DATE    AVAILABLE    AGE
webapp  0/1      1              0             5s
cloud_user_p_7290e748@cloudshell:~ (playground-s-11-5e9e7aff) $ kubectl get deployments
NAME    READY    UP-TO-DATE    AVAILABLE    AGE
webapp  1/1      1              1            20s
cloud_user_p_7290e748@cloudshell:~ (playground-s-11-5e9e7aff) $ kubectl get pods
NAME                                READY    STATUS    RESTARTS    AGE
webapp-77877485b6-zw68d            1/1     Running    0            29s
cloud_user_p_7290e748@cloudshell:~ (playground-s-11-5e9e7aff) $ kubectl describe deployment webapp
Name:
Namespace:
CreationTimestamp:
Labels:
Annotations:
Selector:
Replicas:
StrategyType:
MinReadySeconds:
RollingUpdateStrategy:
Pod Template:
Labels: app=webapp
```

```
Containers:
  nginx:
    Image:          nginx:1.25
    Port:           <none>
    Host Port:      <none>
    Environment:    <none>
    Mounts:         <none>
    Volumes:        <none>
    Node-Selectors:  <none>
    Tolerations:    <none>
Conditions:
  Type           Status  Reason
  ----           -
  Available      True    MinimumReplicasAvailable
  Progressing    True    NewReplicaSetAvailable
OldReplicaSets: <none>
NewReplicaSet:  webapp-77877485b6 (1/1 replicas created)
Events:
  Type           Reason             Age   From               Message
  ----           -
  Normal        ScalingReplicaSet   41s   deployment-controller Scaled up replica set webapp-77877485b6 from 0 to 1
```

```
cloud_user_p_7290e748@cloudshell:~ (playground-s-11-5e9e7aff)$ kubectl set image deployment/webapp nginx=nginx:1.26
deployment.apps/webapp image updated
cloud_user_p_7290e748@cloudshell:~ (playground-s-11-5e9e7aff)$ kubectl rollout status deployment/webapp
deployment "webapp" successfully rolled out
cloud_user_p_7290e748@cloudshell:~ (playground-s-11-5e9e7aff)$ kubectl rollout history deployment/webapp
deployment.apps/webapp
REVISION  CHANGE-CAUSE
1          <none>
2          <none>

cloud_user_p_7290e748@cloudshell:~ (playground-s-11-5e9e7aff)$ kubectl rollout undo deployment/webapp
deployment.apps/webapp rolled back
cloud_user_p_7290e748@cloudshell:~ (playground-s-11-5e9e7aff)$ kubectl rollout status deployment/webapp
deployment "webapp" successfully rolled out
cloud_user_p_7290e748@cloudshell:~ (playground-s-11-5e9e7aff)$ kubectl describe deployment webapp
Name:          webapp
Namespace:     default
CreationTimestamp: Thu, 04 Jun 2026 06:59:43 +0000
Labels:        app=webapp
Annotations:    deployment.kubernetes.io/revision: 3
Selector:      app=webapp
Replicas:      1 desired | 1 updated | 1 total | 1 available | 0 unavailable
StrategyType:  RollingUpdate
MinReadySeconds: 0
RollingUpdateStrategy: 25% max unavailable, 25% max surge
Pod Template:
  Labels:  app=webapp
  Containers:
    nginx:
      Image:          nginx:1.25
      Port:           <none>
      Host Port:      <none>
  Environment:    <none>
  Mounts:         <none>
  Volumes:        <none>
  Node-Selectors:  <none>
  Tolerations:    <none>
Conditions:
  Type           Status  Reason
  ----           -
  Available      True    MinimumReplicasAvailable
  Progressing    True    NewReplicaSetAvailable
OldReplicaSets: webapp-6f754749dc (0/0 replicas created)
NewReplicaSet:  webapp-77877485b6 (1/1 replicas created)
Events:
  Type           Reason             Age   From               Message
  ----           -
  Normal        ScalingReplicaSet   48s   deployment-controller Scaled up replica set webapp-6f754749dc from 0 to 1
  Normal        ScalingReplicaSet   40s   deployment-controller Scaled down replica set webapp-77877485b6 from 1 to 0
  Normal        ScalingReplicaSet   17s (x2 over 3m1s) deployment-controller Scaled up replica set webapp-77877485b6 from 0 to 1
  Normal        ScalingReplicaSet   16s   deployment-controller Scaled down replica set webapp-6f754749dc from 1 to 0
cloud_user_p_7290e748@cloudshell:~ (playground-s-11-5e9e7aff)$ kubectl rollout history deployment/webapp
deployment.apps/webapp
REVISION  CHANGE-CAUSE
2          <none>
3          <none>

cloud_user_p_7290e748@cloudshell:~ (playground-s-11-5e9e7aff)$ kubectl delete deployment webapp
deployment.apps "webapp" deleted from default namespace
```

Task 19 — Kubernetes Namespaces + Multi-Team Environments

This is one of those topics that seems simple at first, but in enterprise environments it becomes absolutely critical.

Real-World Scenario - Imagine 3 teams using one cluster: Frontend Team, Backend Team, Data Team

If everyone deploys: app, you get naming conflicts. Example:

frontend → deployment/app | backend → deployment/app | data → deployment/app

Kubernetes won't know which one you mean.

Solution :

- ❖ Go to Kubernetes, Create cluster with e2-small config, Connect to kubectl and verify.
- ❖ View existing namespaces - `kubectl get namespaces`
- ❖ Create Dev Namespace - `kubectl create namespace dev`
Create QA Namespace - `kubectl create namespace qa`
Create Prod Namespace - `kubectl create namespace prod`
Verify all : `kubectl get namespaces`
- ❖ Deploy app in dev - `kubectl create deployment app --image=nginx -n dev`
Deploy app in qa - `kubectl create deployment app --image=nginx -n qa`
Deploy app in prod - `kubectl create deployment app --image=nginx -n prod`
- ❖ View deployments - `kubectl get deployments -A` [-A means all namespaces]
View only dev - `kubectl get deployments -n dev`
View only qa - `kubectl get deployments -n qa`
View only prod - `kubectl get deployments -n prod`
- ❖ Set default namespace - `kubectl config set-context --current --namespace=dev`
Verify: `kubectl get pods`
- ❖ Check current namespace - `kubectl config view --minify | grep namespace`
- ❖ Switch to qa - `kubectl config set-context --current --namespace=qa`, Verify it.
- ❖ Switch back to default - `kubectl config set-context --current --namespace=default`
- ❖ Cleanup: `kubectl delete namespace dev` | `kubectl delete namespace qa` | `kubectl delete namespace prod`
Then delete the cluster.

Concepts :

Namespaces provide logical isolation within a Kubernetes cluster, enabling multiple teams, applications, and environments to share cluster resources while maintaining organizational, security, and operational boundaries.

View Existing namespace – You'll see something like : default | kube-system | kube-public | kube-node-lease

What Are These? – default : Where resources go if you don't specify a namespace.

kube-system - Stores Kubernetes system components. Examples: DNS, Metrics Server, Controllers

kube-public - Publicly readable resources. Rarely used.

All three deployments are named: app, Yet Kubernetes allows it. Why? Because:

dev/app | qa/app | prod/app, are different resources. Namespaces act like folders. Think:

Windows

- Dev Folder
- QA Folder
- Prod Folder

Kubernetes Namespaces work similarly.

Sandbox Screenshots :


```

Welcome to Cloud Shell! Type "help" to get started, or type "gemini" to try prompting with Gemini CLI.
Your Cloud Platform project in this session is set to playground-s-11-fab9ae9c.
Use 'gcloud config set project [PROJECT_ID]' to change to a different project.
gcloud container clusters get-credentials namespace-cluster --zone us-central1-a --project playground-s-11-fab9ae9ccloud_user_p_5db3e380@cloudshell:~ (playground-s-11-fab9ae9c)$ gcloud container clusters get-credentials namespace-cluster --zone us-central1-a --project playground-s-11-fab9ae9c
Fetching cluster endpoint and auth data.
kubeconfig entry generated for namespace-cluster.
cloud_user_p_5db3e380@cloudshell:~ (playground-s-11-fab9ae9c)$ kubectl get nodes
NAME                                STATUS    ROLES    AGE    VERSION
gke-namespace-cluster-default-pool-19d43c61-f3fx  Ready    <none>   84s    v1.35.3-gke.1389002
gke-namespace-cluster-default-pool-19d43c61-jklx  Ready    <none>   82s    v1.35.3-gke.1389002
gke-namespace-cluster-default-pool-19d43c61-vb1x  Ready    <none>   83s    v1.35.3-gke.1389002
cloud_user_p_5db3e380@cloudshell:~ (playground-s-11-fab9ae9c)$ kubectl get namespaces
NAME                                STATUS    AGE
default                            Active    3m3s
gke-managed-cim                    Active    2m34s
gke-managed-networking-dra-driver  Active    95s
gke-managed-system                 Active    2m15s
gke-managed-volume-populator       Active    2m10s
gmp-public                         Active    111s
gmp-system                         Active    111s
kube-node-lease                    Active    3m3s
kube-public                        Active    3m3s
kube-system                        Active    3m4s
cloud_user_p_5db3e380@cloudshell:~ (playground-s-11-fab9ae9c)$ kubectl create namespace dev
namespace/dev created
cloud_user_p_5db3e380@cloudshell:~ (playground-s-11-fab9ae9c)$ kubectl create namespace qa
namespace/qa created
cloud_user_p_5db3e380@cloudshell:~ (playground-s-11-fab9ae9c)$ kubectl create namespace prod
namespace/prod created

```

```

cloud_user_p_5db3e380@cloudshell:~ (playground-s-11-fab9ae9c)$ kubectl get namespaces
NAME                                STATUS    AGE
default                            Active    3m36s
dev                                Active    18s
gke-managed-cim                    Active    3m7s
gke-managed-networking-dra-driver  Active    2m8s
gke-managed-system                 Active    2m48s
gke-managed-volume-populator       Active    2m43s
gmp-public                         Active    2m24s
gmp-system                         Active    2m24s
kube-node-lease                    Active    3m36s
kube-public                        Active    3m36s
kube-system                        Active    3m37s
prod                               Active    6s
qa                                 Active    12s
cloud_user_p_5db3e380@cloudshell:~ (playground-s-11-fab9ae9c)$ kubectl create deployment app \
--image=nginx \
-n dev
deployment.apps/app created
cloud_user_p_5db3e380@cloudshell:~ (playground-s-11-fab9ae9c)$ kubectl create deployment app \
--image=nginx \
-n qa
deployment.apps/app created
cloud_user_p_5db3e380@cloudshell:~ (playground-s-11-fab9ae9c)$ kubectl create deployment app \
--image=nginx \
-n prod
deployment.apps/app created

```

```
cloud_user_p_5db3e380@cloudshell:~ (playground-s-11-fab9ae9c)$ kubectl get deployments -A
NAMESPACE   NAME                                READY   UP-TO-DATE   AVAILABLE   AGE
dev          app                                1/1     1             1           26s
gmp-system   gmp-operator                       1/1     1             1           2m57s
gmp-system   rule-evaluator                     0/0     0             0           2m57s
kube-system  event-exporter-gke                 1/1     1             1           3m34s
kube-system  konnectivity-agent                 3/3     3             3           3m12s
kube-system  konnectivity-agent-autoscaler      1/1     1             1           3m10s
kube-system  kube-dns                           2/2     2             2           3m56s
kube-system  kube-dns-autoscaler                1/1     1             1           3m8s
kube-system  17-default-backend                 1/1     1             1           3m5s
kube-system  metrics-server-v1.35.1             1/1     1             1           2m50s
prod         app                                1/1     1             1           14s
qa           app                                1/1     1             1           20s

cloud_user_p_5db3e380@cloudshell:~ (playground-s-11-fab9ae9c)$ kubectl get deployments -n dev
NAME   READY   UP-TO-DATE   AVAILABLE   AGE
app    1/1     1             1           38s

cloud_user_p_5db3e380@cloudshell:~ (playground-s-11-fab9ae9c)$ kubectl get deployments -n qa
NAME   READY   UP-TO-DATE   AVAILABLE   AGE
app    1/1     1             1           40s

cloud_user_p_5db3e380@cloudshell:~ (playground-s-11-fab9ae9c)$ kubectl get deployments -n prod
NAME   READY   UP-TO-DATE   AVAILABLE   AGE
app    1/1     1             1           40s

cloud_user_p_5db3e380@cloudshell:~ (playground-s-11-fab9ae9c)$ kubectl config set-context --current --namespace=dev
Context "gke_playground-s-11-fab9ae9c_us-central1-a_namespace-cluster" modified.
cloud_user_p_5db3e380@cloudshell:~ (playground-s-11-fab9ae9c)$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
app-6b97cd8cbd-kbjbp                1/1     Running   0           72s
```

```
cloud_user_p_5db3e380@cloudshell:~ (playground-s-11-fab9ae9c)$ kubectl config view --minify | grep namespace
name: gke_playground-s-11-fab9ae9c_us-central1-a_namespace-cluster
cluster: gke_playground-s-11-fab9ae9c_us-central1-a_namespace-cluster
namespace: dev
user: gke_playground-s-11-fab9ae9c_us-central1-a_namespace-cluster
name: gke_playground-s-11-fab9ae9c_us-central1-a_namespace-cluster
current-context: gke_playground-s-11-fab9ae9c_us-central1-a_namespace-cluster
- name: gke_playground-s-11-fab9ae9c_us-central1-a_namespace-cluster
cloud_user_p_5db3e380@cloudshell:~ (playground-s-11-fab9ae9c)$ kubectl config set-context --current --namespace=qa
Context "gke_playground-s-11-fab9ae9c_us-central1-a_namespace-cluster" modified.
cloud_user_p_5db3e380@cloudshell:~ (playground-s-11-fab9ae9c)$ kubectl config view --minify | grep namespace
name: gke_playground-s-11-fab9ae9c_us-central1-a_namespace-cluster
cluster: gke_playground-s-11-fab9ae9c_us-central1-a_namespace-cluster
namespace: qa
user: gke_playground-s-11-fab9ae9c_us-central1-a_namespace-cluster
name: gke_playground-s-11-fab9ae9c_us-central1-a_namespace-cluster
current-context: gke_playground-s-11-fab9ae9c_us-central1-a_namespace-cluster
- name: gke_playground-s-11-fab9ae9c_us-central1-a_namespace-cluster
cloud_user_p_5db3e380@cloudshell:~ (playground-s-11-fab9ae9c)$ kubectl config set-context --current --namespace=default
Context "gke_playground-s-11-fab9ae9c_us-central1-a_namespace-cluster" modified.
```

Task 20 — Resource Requests, Limits & Quotas

Kubernetes allows you to define: Requests (Minimum resources a Pod needs)

Limits (Maximum resources a Pod can consume), Quotas (Limits for an entire Namespace or Team)

Solution :

- Go to Kubernetes, create cluster with e2-small config, Connect to kubectl and verify it.
- Create namespace - `kubectl create namespace dev`, Verify it: `kubectl get ns`
- Create resource limited deployment - `nano resources.yaml` , paste this :
apiVersion: apps/v1
kind: Deployment

```
metadata:  
  name: nginx-resources  
  namespace: dev
```

```
spec:  
  replicas: 1
```

```
selector:  
  matchLabels:  
    app: nginx
```

```
template:  
  metadata:  
    labels:  
      app: nginx
```

```
spec:  
  containers:  
    - name: nginx  
      image: nginx
```

```
resources:  
  requests:  
    cpu: "100m"  
    memory: "128Mi"
```

```
limits:  
  cpu: "200m"  
  memory: "256Mi"
```

Save it: Ctrl O + Enter + Ctrl X

- Deploy - `kubectl apply -f resources.yaml`, verify: `kubectl get deployments -n dev`
- Inspect resources - `kubectl describe deployment nginx-resources -n dev`
- Create namespace quota - `nano quota.yaml`, paste this:
apiVersion: v1
kind: ResourceQuota

metadata:

name: dev-quota

namespace: dev

spec:

hard:

requests.cpu: "1"

requests.memory: 1Gi

limits.cpu: "2"

limits.memory: 2Gi

pod: "10"

Save it: Ctrl O + Enter + Ctrl X

- Apply quota - `kubectl apply -f quota.yaml`, verify: `kubectl get resourcequota -n dev`
Detailed view - `kubectl describe resourcequota dev-quota -n dev` (Observe current usage)
- Check pod - `kubectl get pods -n dev`,
`kubectl describe pod <pod-name> -n dev` [replace pod name with yours]

Concepts :

Kubernetes resource requests reserve CPU and memory for a Pod, while limits prevent a Pod from consuming excessive resources. ResourceQuotas enforce namespace-wide resource consumption limits, helping organizations share cluster resources fairly across teams and environments.

cpu: "100m", means: 100 milliCPU = 0.1 CPU Core. Examples: 1000m = 1 CPU , 500m = 0.5 CPU , 250m = 0.25 CPU

Memory: 128Mi, means: 128 MB approximately

Used CPU: 100m, Hard CPU: 1 - means: 10% consumed, 90% available

Sandbox screenshots :

```
Welcome to Cloud Shell! Type "help" to get started, or type "gemini" to try prompting with Gemini CLI.
Your Cloud Platform project in this session is set to playground-s-11-590da2e7.
Use 'gcloud config set project [PROJECT_ID]' to change to a different project.
gcloud container clusters get-credentials resource-cluster --zone us-central1-a --project playground-s-11-590da2e7cloud_user_p_5883523a@cloudshell:~ (playground-s-11-590da2e7)$ gcloud container clusters get-credentials resource-cluster --zone us-central1-a --project playground-s-11-590da2e7
Fetching cluster endpoint and auth data.
kubeconfig entry generated for resource-cluster.
cloud_user_p_5883523a@cloudshell:~ (playground-s-11-590da2e7)$ kubectl get nodes
NAME                                STATUS    ROLES    AGE   VERSION
gke-resource-cluster-default-pool-06b9d397-4mlm   Ready    <none>   92s   v1.35.3-gke.2190000
gke-resource-cluster-default-pool-06b9d397-r33n   Ready    <none>   93s   v1.35.3-gke.2190000
gke-resource-cluster-default-pool-06b9d397-z138   Ready    <none>   92s   v1.35.3-gke.2190000
cloud_user_p_5883523a@cloudshell:~ (playground-s-11-590da2e7)$ kubectl create namespace dev
namespace/dev created
cloud_user_p_5883523a@cloudshell:~ (playground-s-11-590da2e7)$ kubectl get ns
NAME                                STATUS    AGE
default                             Active    3m20s
dev                                  Active    5s
gke-managed-cim                      Active    2m56s
gke-managed-networking-dra-driver    Active    116s
gke-managed-system                   Active    2m37s
gke-managed-volume-populator         Active    2m32s
gmp-public                           Active    2m12s
gmp-system                           Active    2m12s
kube-node-lease                      Active    3m20s
kube-public                           Active    3m20s
kube-system                           Active    3m20s
cloud_user_p_5883523a@cloudshell:~ (playground-s-11-590da2e7)$ nano resources.yaml
cloud_user_p_5883523a@cloudshell:~ (playground-s-11-590da2e7)$ kubectl apply -f resources.yaml
deployment.apps/nginx-resources created
cloud_user_p_5883523a@cloudshell:~ (playground-s-11-590da2e7)$ kubectl get deployments -n dev
NAME                                READY    UP-TO-DATE    AVAILABLE    AGE
nginx-resources                      1/1      1              1             15s
```

```
cloud_user_p_5883523a@cloudshell:~ (playground-s-11-590da2e7)$ kubectl describe deployment nginx-resources -n dev
Name: nginx-resources
Namespace: dev
CreationTimestamp: Sun, 07 Jun 2026 07:52:36 +0000
Labels: <none>
Annotations: deployment.kubernetes.io/revision: 1
Selector: app=nginx
Replicas: 1 desired | 1 updated | 1 total | 1 available | 0 unavailable
StrategyType: RollingUpdate
MinReadySeconds: 0
RollingUpdateStrategy: 25% max unavailable, 25% max surge
Pod Template:
  Labels: app=nginx
  Containers:
    nginx:
      Image: nginx
      Port: <none>
      Host Port: <none>
      Limits:
        cpu: 200m
        memory: 256Mi
      Requests:
        cpu: 100m
        memory: 128Mi
      Environment: <none>
      Mounts: <none>
      Volumes: <none>
      Node-Selectors: <none>
      Tolerations: <none>
Conditions:
  Type           Status  Reason
  ----           -
  Available      True    MinimumReplicasAvailable
  Progressing    True    NewReplicaSetAvailable
OldReplicaSets: <none>
```

```
OldReplicaSets: <none>
NewReplicaSet: nginx-resources-65f54b4ccf (1/1 replicas created)
Events:
  Type    Reason             Age    From          Message
  ----    -
  Normal  ScalingReplicaSet  26s    deployment-controller  Scaled up replica set nginx-resources-65f54b4ccf from 0 to 1
cloud_user_p_5883523a@cloudshell:~ (playground-s-11-590da2e7)$ nano quota.yaml
cloud_user_p_5883523a@cloudshell:~ (playground-s-11-590da2e7)$ kubectl apply -f quota.yaml
resourcequota/dev-quota created
cloud_user_p_5883523a@cloudshell:~ (playground-s-11-590da2e7)$ kubectl get resourcequota -n dev
NAME          REQUEST                                     LIMIT                                     AGE
dev-quota     pods: 1/10, requests.cpu: 100m/1, requests.memory: 128Mi/1Gi  limits.cpu: 200m/2, limits.memory: 256Mi/2Gi  8s
cloud_user_p_5883523a@cloudshell:~ (playground-s-11-590da2e7)$ kubectl describe resourcequota dev-quota -n dev
Name: dev-quota
Namespace: dev
Resource      Used  Hard
-----
limits.cpu    200m  2
limits.memory 256Mi 2Gi
pods          1     10
requests.cpu   100m  1
requests.memory 128Mi 1Gi
cloud_user_p_5883523a@cloudshell:~ (playground-s-11-590da2e7)$ kubectl get pods -n dev
NAME                                READY  STATUS  RESTARTS  AGE
nginx-resources-65f54b4ccf-zb44h    1/1    Running  0          3m3s
```

```
cloud_user_p_5883523a@cloudshell:~ (playground-s-11-590da2e7) $ kubectl describe pod nginx-resources-65f54b4ccf-zb44h -n dev
Name:          nginx-resources-65f54b4ccf-zb44h
Namespace:     dev
Priority:       0
Service Account: default
Node:          gke-resource-cluster-default-pool-06b9d397-4mlm/10.128.0.3
Start Time:    Sun, 07 Jun 2026 07:52:36 +0000
Labels:        app=nginx
               pod-template-hash=65f54b4ccf
               topology.kubernetes.io/region=us-central1
               topology.kubernetes.io/zone=us-central1-a
Annotations:   <none>
Status:        Running
IP:            10.115.128.4
IPs:
  IP:          10.115.128.4
Controlled By: ReplicaSet/nginx-resources-65f54b4ccf
Containers:
  nginx:
    Container ID:  containerd://d431894a2ac68db925ba6120b5df31be6036b40d6b492a49ac62882777ee67f6
    Image:         nginx
    Image ID:      docker.io/library/nginx@sha256:5aca99593157f4ae539a5dec1092a0ad8762f8e2eb1789085a13a0f5622369f6
    Port:         <none>
    Host Port:    <none>
    State:        Running
      Started:    Sun, 07 Jun 2026 07:52:42 +0000
    Ready:        True
    Restart Count: 0
    Limits:
      cpu:        200m
      memory:     256Mi
    Requests:
      cpu:        100m
      memory:     128Mi
    Environment:  <none>
```

```
Mounts:
  /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-dk67q (ro)
Conditions:
  Type              Status
  PodReadyToStartContainers  True
  Initialized        True
  Ready              True
  ContainersReady    True
  PodScheduled       True
Volumes:
  kube-api-access-dk67q:
    Type:          Projected (a volume that contains injected data from multiple sources)
    TokenExpirationSeconds: 3607
    ConfigMapName:  kube-root-ca.crt
    Optional:      false
    DownwardAPI:   true
QoS Class:        Burstable
Node-Selectors:   <none>
Tolerations:      node.kubernetes.io/not-ready:NoExecute op=Exists for 300s
                  node.kubernetes.io/unreachable:NoExecute op=Exists for 300s
Events:
  Type    Reason      Age    From          Message
  ----    -
  Normal  Scheduled   3m51s  default-scheduler  Successfully assigned dev/nginx-resources-65f54b4ccf-zb44h to gke-resource-cluster-default-pool-06b9d397-4mlm
  Normal  Pulling     3m50s  kubelet        spec.containers{nginx}: Pulling image "nginx"
  Normal  Pulled      3m45s  kubelet        spec.containers{nginx}: Successfully pulled image "nginx" in 5.018s (5.018s including waiting). Image size: 63120520 bytes
  Normal  Created     3m45s  kubelet        spec.containers{nginx}: Container created
  Normal  Started     3m45s  kubelet        spec.containers{nginx}: Container started
```

Task 21 — Horizontal Pod Autoscaler (HPA)

This is one of the features that makes Kubernetes feel "smart." Up until now, when you wanted more Pods, you manually ran: `kubectl scale deployment webapp --replicas=4`. But in production, nobody sits there watching CPU graphs all day.

Instead: Traffic increases → CPU rises → Kubernetes detects it → More Pods created automatically. That's HPA.

Solution:

- Go to Kubernetes, create cluster e2-small config, connect to kubectl and verify.
- Create deployment: `kubectl create deployment webapp --image=nginx`. And verify.
- Expose deployment: `kubectl expose deployment webapp --port=80 --target-port=80`.
Verify it: `kubectl get svc`
- Verify metrics server: `kubectl top nodes`
- Create Autoscaler: `kubectl autoscale deployment webapp --cpu-percent=50 --min=1 --max=5`
Verify hpa: `kubectl get hpa`, watch hpa: `kubectl get hpa -w`
- Check deployment scale: `kubectl get deployment webapp`
- Describe autoscaler: `kubectl describe hpa webapp`

[Note: Horizontal Pod Autoscaler (HPA) was successfully created, but automatic scaling did not occur because the deployment was created without CPU resource requests (see step 2). HPA calculates utilization as a percentage of the requested CPU resources, and without a defined CPU request, it cannot determine scaling metrics. This resulted in the error `FailedGetResourceMetric: missing request for cpu`, highlighting the dependency of HPA on properly configured resource requests.

Do We Need to Fix It? Not really. For learning purposes, we learned quite enough:

HPA created successfully | HPA controller running | Metrics available | Identified why scaling failed

That's actually more educational than it working immediately]

Concepts:

Horizontal Pod Autoscaler (HPA) monitors resource utilization, typically CPU usage, and automatically increases or decreases the number of Pod replicas to maintain the target utilization level. This enables applications to scale according to workload demand without manual intervention.

Understanding autoscaler command - If CPU > 50% - Scale Up, Minimum Pods = 1 | Maximum Pods = 5

Sandbox Screenshots:

```
Welcome to Cloud Shell! Type "help" to get started, or type "gemini" to try prompting with Gemini CLI.
Your Cloud Platform project in this session is set to playground-s-11-007bd095.
Use 'gcloud config set project [PROJECT_ID]' to change to a different project.
gcloud container clusters get-credentials hpa-cluster --zone us-central1-a --project playground-s-11-007bd095 cloud_user_p_6fa9e712@cloudshell:~ (playground-s-11-007bd095) $ gcloud container cl
usters get-credentials hpa-cluster --zone us-central1-a --project playground-s-11-007bd095
Fetching cluster endpoint and auth data.
kubeconfig entry generated for hpa-cluster.
cloud_user_p_6fa9e712@cloudshell:~ (playground-s-11-007bd095) $ kubectl get nodes
NAME                                STATUS    ROLES    AGE   VERSION
gke-hpa-cluster-default-pool-75f1ffcl-3p7t    Ready    <none>    112s   v1.35.3-gke.2190000
gke-hpa-cluster-default-pool-75f1ffcl-792n    Ready    <none>    107s   v1.35.3-gke.2190000
gke-hpa-cluster-default-pool-75f1ffcl-kwzq    Ready    <none>    111s   v1.35.3-gke.2190000
cloud_user_p_6fa9e712@cloudshell:~ (playground-s-11-007bd095) $ kubectl create deployment webapp --image=nginx
deployment.apps/webapp created
cloud_user_p_6fa9e712@cloudshell:~ (playground-s-11-007bd095) $ kubectl get deployments
NAME    READY   UP-TO-DATE   AVAILABLE   AGE
webapp   0/1     1             0            5s
cloud_user_p_6fa9e712@cloudshell:~ (playground-s-11-007bd095) $ kubectl expose deployment webapp \
--port=80 \
--target-port=80
service/webapp exposed
cloud_user_p_6fa9e712@cloudshell:~ (playground-s-11-007bd095) $ kubectl get svc
NAME            TYPE        CLUSTER-IP    EXTERNAL-IP   PORT(S)    AGE
kubernetes      ClusterIP   34.118.224.1   <none>        443/TCP    3m47s
webapp          ClusterIP   34.118.233.18  <none>        80/TCP     6s
cloud_user_p_6fa9e712@cloudshell:~ (playground-s-11-007bd095) $ kubectl top nodes
NAME                                CPU(cores)  CPU(%)  MEMORY(bytes)  MEMORY(%)
gke-hpa-cluster-default-pool-75f1ffcl-3p7t    594m        63%     1251Mi         92%
gke-hpa-cluster-default-pool-75f1ffcl-792n    445m        47%     1137Mi         83%
gke-hpa-cluster-default-pool-75f1ffcl-kwzq    237m        25%     1105Mi         81%
```



```

cloud_user_p_6fa9e712@cloudshell:~ (playground-s-11-007bd095)$ kubectl autoscale deployment webapp \
--cpu-percent=50 \
--min=1 \
--max=5
Flag --cpu-percent has been deprecated, Use --cpu with percentage or resource quantity format (e.g., '70%' for utilization or '500m' for millicPU).
horizontalpodautoscaler.autoscaling/webapp autoscaled
cloud_user_p_6fa9e712@cloudshell:~ (playground-s-11-007bd095)$ kubectl get hpa
NAME      REFERENCE            TARGETS      MINPODS  MAXPODS  REPLICAS  AGE
webapp    Deployment/webapp     cpu: <unknown>/50%  1        5        1         17s
cloud_user_p_6fa9e712@cloudshell:~ (playground-s-11-007bd095)$ kubectl get hpa -w
NAME      REFERENCE            TARGETS      MINPODS  MAXPODS  REPLICAS  AGE
webapp    Deployment/webapp     cpu: <unknown>/50%  1        5        1         34s
^Ccloud_user_p_6fa9e712@cloudshell:~ (playground-s-11-007bd095)$ kubectl get deployment webapp
NAME      READY   UP-TO-DATE   AVAILABLE   AGE
webapp    1/1     1            1           98s
cloud_user_p_6fa9e712@cloudshell:~ (playground-s-11-007bd095)$ kubectl describe hpa webapp
Name:
Namespace:
Labels:
Annotations:
CreationTimestamp:
Reference:
Metrics:
  resource cpu on pods  (as a percentage of request):  <unknown> / 50%
Min replicas:
Max replicas:
Deployment pods:
Conditions:
  Type             Status  Reason
  ----             -
  AbleToScale      True    ReadyForNewScale
  ScalingActive    False   FailedGetResourceMetric
  ScalingLimited   False   DesiredWithinRange
  Message
  -----
  recommended size matches current size
  the HPA was unable to compute the replica count: missing request for cpu
  the desired count is within the acceptable range
Events:
Events:
  Type             Reason
  ----             -
  Warning          FailedGetResourceMetric
  Warning          FailedGetResourceMetric
  Age              From
  ----              -
  41s (x2 over 45s) horizontal-pod-autoscaler
  15s (x3 over 41s) horizontal-pod-autoscaler
  Message
  -----
  No recommendation
  missing request for cpu

```

Task 22 — Helm: The Kubernetes Package Manager

Before Helm - Suppose you want to deploy: Prometheus You might need: 6+ YAML files. For bigger applications: 100+ YAML files Not fun.

Helm Solution - Think of Helm as: apt-get for Ubuntu | npm for Node.js | pip for Python, but for Kubernetes.

Instead of: Write 20 YAML files, you run: helm install myapp, and Helm creates everything.

Solution :

- ❖ Go to Kubernetes, create cluster with e2-small config, connect to kubectl and verify.
- ❖ Check helm - helm version
- ❖ Add helm repo - helm repo add bitnami <https://charts.bitnami.com/bitnami>
Update repo -helm repo update
- ❖ Search charts - helm search repo nginx
- ❖ Install nginx chart - helm install my-nginx bitnami/nginx , verify - helm list
Verify Kubernetes resources - kubectl get all
- ❖ Inspect release - helm status my-nginx , upgrade release - helm upgrade my-nginx bitnami/nginx
View release history - helm history my-nginx
- ❖ Rollback - helm rollback my-nginx 1, verify - helm history my-nginx
- ❖ Uninstall - helm uninstall my-nginx , verify - helm list | kubectl get all

Concepts –

Helm packages Kubernetes resources into reusable charts, allowing applications to be installed, upgraded, rolled back, and removed using simple commands. It reduces manual YAML management and is widely used to deploy complex applications in Kubernetes environments.

What Is Helm? Helm consists of: Chart & Release :

Chart - The package. Example: nginx, Prometheus etc.

Release - An installed instance of a chart. Example: nginx-prod, nginx-dev | Both can come from the same chart.

Understanding Charts - Docker Hub → Container Images | Helm Repository → Charts

Understanding Install nginx chart command - Helm automatically creates:

Deployment, Service, Pods, Labels, Selectors without you writing YAML.

Sandbox screenshots –

```
Welcome to Cloud Shell! Type "help" to get started, or type "gemini" to try prompting with Gemini CLI.
Your Cloud Platform project in this session is set to playground-s-11-33c2odd4.
Use 'gcloud config set project [PROJECT_ID]' to change to a different project.
gcloud container clusters get-credentials helm-cluster --zone us-central1-a --project playground-s-11-33c2odd4cloud_user_p_a5e85419@cloudshell:~ (playground-s-11-33c2odd4) $ gcloud container c
usters get-credentials helm-cluster --zone us-central1-a --project playground-s-11-33c2odd4
Fetching cluster endpoint and auth data.
kubeconfig entry generated for helm-cluster.
cloud_user_p_a5e85419@cloudshell:~ (playground-s-11-33c2odd4) $ kubectl get nodes
NAME                                STATUS    ROLES    AGE   VERSION
gke-helm-cluster-default-pool-4759e435-4bmg Ready    <none>   91s   v1.35.3-gke.2190000
gke-helm-cluster-default-pool-4759e435-59z3 Ready    <none>   92s   v1.35.3-gke.2190000
gke-helm-cluster-default-pool-4759e435-h3sh Ready    <none>   89s   v1.35.3-gke.2190000
cloud_user_p_a5e85419@cloudshell:~ (playground-s-11-33c2odd4) $ helm version
version.BuildInfo{Version:"v4.1.4", GitCommit:"05fa37973dc9e42b76e1d2883494c87174b6074f", GitTreeState:"clean", GoVersion:"go1.25.9", KubeClientVersion:"v1.35"}
cloud_user_p_a5e85419@cloudshell:~ (playground-s-11-33c2odd4) $ helm repo add bitnami https://charts.bitnami.com/bitnami
"bitnami" has been added to your repositories
cloud_user_p_a5e85419@cloudshell:~ (playground-s-11-33c2odd4) $ helm repo update
Hang tight while we grab the latest from your chart repositories...
...Successfully got an update from the "bitnami" chart repository
Update Complete. ✨Happy Helming!✨
cloud_user_p_a5e85419@cloudshell:~ (playground-s-11-33c2odd4) $ helm search repo nginx
NAME                                CHART VERSION   APP VERSION     DESCRIPTION
bitnami/nginx                       25.0.3          1.31.1          NGINX Open Source is a web server that can be a...
bitnami/nginx-ingress-controller    12.0.7          1.13.1          NGINX Ingress Controller is an Ingress controll...
bitnami/nginx-intel                 2.1.15          0.4.9           DEPRECATED NGINX Open Source for Intel is a lig...
cloud_user_p_a5e85419@cloudshell:~ (playground-s-11-33c2odd4) $ helm install my-nginx bitnami/nginx
NAME: my-nginx
LAST DEPLOYED: Thu Jun 11 07:10:33 2026
NAMESPACE: default
STATUS: deployed
REVISION: 1
DESCRIPTION: Install complete
TEST SUITE: None
NOTES:
CHART NAME: nginx
CHART VERSION: 25.0.3
```

```
CHART VERSION: 25.0.3
APP VERSION: 1.31.1

⚠ WARNING: Since August 28th, 2025, only a limited subset of images/charts are available for free.
  Subscribe to Bitnami Secure Images to receive continued support and security updates.
  More info at https://bitnami.com and https://github.com/bitnami/containers/issues/83267

** Please be patient while the chart is being deployed **
NGINX can be accessed through the following DNS name from within your cluster:

    my-nginx.default.svc.cluster.local (port 80)

To access NGINX from outside the cluster, follow the steps below:

1. Get the NGINX URL by running these commands:

NOTE: It may take a few minutes for the LoadBalancer IP to be available.
  Watch the status with: 'kubectl get svc --namespace default -w my-nginx'

export SERVICE_PORT=$(kubectl get --namespace default -o jsonpath="{.spec.ports[0].port}" services my-nginx)
export SERVICE_IP=$(kubectl get svc --namespace default my-nginx -o jsonpath='{.status.loadBalancer.ingress[0].ip}')
echo "http://${SERVICE_IP}:${SERVICE_PORT}"

WARNING: Rolling tag detected (bitnami/nginx:latest), please note that it is strongly recommended to avoid using rolling tags in a production environment.
+info https://techdocs.broadcom.com/us/en/vmware-tanzu/bitnami-secure-images/bitnami-secure-images/services/bsi-doc/apps-tutorials-understand-rolling-tags-containers-index.html
WARNING: Rolling tag detected (bitnami/git:latest), please note that it is strongly recommended to avoid using rolling tags in a production environment.
+info https://techdocs.broadcom.com/us/en/vmware-tanzu/bitnami-secure-images/bitnami-secure-images/services/bsi-doc/apps-tutorials-understand-rolling-tags-containers-index.html
WARNING: Rolling tag detected (bitnami/nginx-exporter:latest), please note that it is strongly recommended to avoid using rolling tags in a production environment.
+info https://techdocs.broadcom.com/us/en/vmware-tanzu/bitnami-secure-images/bitnami-secure-images/services/bsi-doc/apps-tutorials-understand-rolling-tags-containers-index.html

WARNING: There are "resources" sections in the chart not set. Using "resourcesPresets" is not recommended for production. For production installations, please set the following values according to your workload needs:
- cloneStaticSiteFromGit.gitSync.resources
- resources
+info https://kubernetes.io/docs/concepts/configuration/manage-resources-containers/
```



```
cloud_user_p_a5e85419@cloudshell:~ (playground-s-11-33c2cdd4)$ helm list
NAME          NAMESPACE    REVISION    UPDATED                               STATUS          CHART          APP VERSION
my-nginx      default       1           2026-06-11 07:10:33.595639166 +0000 UTC deployed        nginx-25.0.3   1.31.1
cloud_user_p_a5e85419@cloudshell:~ (playground-s-11-33c2cdd4)$ kubectl get all
NAME                READY   STATUS    RESTARTS   AGE
pod/my-nginx-6d4484d577-9rtdk  1/1     Running   0           59s

NAME                TYPE          CLUSTER-IP    EXTERNAL-IP    PORT(S)          AGE
service/kubernetes  ClusterIP     34.118.224.1   <none>          443/TCP          5m18s
service/my-nginx     LoadBalancer 34.118.239.231 35.238.156.5   80:31690/TCP,443:32278/TCP 60s

NAME                READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/my-nginx  1/1     1             1           60s

NAME                DESIRED   CURRENT   READY   AGE
replicaset.apps/my-nginx-6d4484d577  1         1         1       61s
cloud_user_p_a5e85419@cloudshell:~ (playground-s-11-33c2cdd4)$ helm status my-nginx
NAME: my-nginx
LAST DEPLOYED: Thu Jun 11 07:10:33 2026
NAMESPACE: default
STATUS: deployed
REVISION: 1
DESCRIPTION: Install complete
RESOURCES:
==> v1/PodDisruptionBudget
NAME          MIN AVAILABLE   MAX UNAVAILABLE   ALLOWED DISRUPTIONS   AGE
my-nginx      N/A             1                 1                     85s

==> v1/ServiceAccount
NAME          AGE
my-nginx      85s

==> v1/Secret
NAME          TYPE          DATA   AGE
my-nginx-tls  kubernetes.io/tls  3       85s
```

```
NAME          TYPE          DATA   AGE
my-nginx-tls  kubernetes.io/tls  3       85s

==> v1/Service
NAME          TYPE          CLUSTER-IP    EXTERNAL-IP    PORT(S)          AGE
my-nginx      LoadBalancer 34.118.239.231 35.238.156.5   80:31690/TCP,443:32278/TCP 85s

==> v1/Deployment
NAME          READY   UP-TO-DATE   AVAILABLE   AGE
my-nginx      1/1     1             1           85s

==> v1/Pod(related)
NAME          READY   STATUS    RESTARTS   AGE
my-nginx-6d4484d577-9rtdk  1/1     Running   0           85s

==> v1/NetworkPolicy
NAME          POD-SELECTOR          AGE
my-nginx      app.kubernetes.io/instance=my-nginx,app.kubernetes.io/name=nginx 86s

TEST SUITE: None
NOTES:
CHART NAME: nginx
CHART VERSION: 25.0.3
APP VERSION: 1.31.1

⚠ WARNING: Since August 28th, 2025, only a limited subset of images/charts are available for free.
Subscribe to Bitnami Secure Images to receive continued support and security updates.
More info at https://bitnami.com and https://github.com/bitnami/containers/issues/83267

** Please be patient while the chart is being deployed **
NGINX can be accessed through the following DNS name from within your cluster:

    my-nginx.default.svc.cluster.local (port 80)

To access NGINX from outside the cluster, follow the steps below:
```

To access NGINX from outside the cluster, follow the steps below:

1. Get the NGINX URL by running these commands:

NOTE: It may take a few minutes for the LoadBalancer IP to be available.
Watch the status with: 'kubectl get svc --namespace default -w my-nginx'

```
export SERVICE_PORT=$(kubectl get --namespace default -o jsonpath="{.spec.ports[0].port}" services my-nginx)
export SERVICE_IP=$(kubectl get svc --namespace default my-nginx -o jsonpath='{.status.loadBalancer.ingress[0].ip}')
echo "http://${SERVICE_IP}:${SERVICE_PORT}"
```

WARNING: Rolling tag detected (bitnami/nginx:latest), please note that it is strongly recommended to avoid using rolling tags in a production environment.
+info https://techdocs.broadcom.com/us/en/vmware-tanzu/bitnami-secure-images/bitnami-secure-images/services/bsi-doc/apps-tutorials-understand-rolling-tags-containers-index.html
WARNING: Rolling tag detected (bitnami/git:latest), please note that it is strongly recommended to avoid using rolling tags in a production environment.
+info https://techdocs.broadcom.com/us/en/vmware-tanzu/bitnami-secure-images/bitnami-secure-images/services/bsi-doc/apps-tutorials-understand-rolling-tags-containers-index.html
WARNING: Rolling tag detected (bitnami/nginx-exporter:latest), please note that it is strongly recommended to avoid using rolling tags in a production environment.
+info https://techdocs.broadcom.com/us/en/vmware-tanzu/bitnami-secure-images/bitnami-secure-images/services/bsi-doc/apps-tutorials-understand-rolling-tags-containers-index.html

WARNING: There are "resources" sections in the chart not set. Using "resourcesPreset" is not recommended for production. For production installations, please set the following values according to your workload needs:

```
- cloneStaticSiteFromGit.gitSync.resources
- resources
```

+info https://kubernetes.io/docs/concepts/configuration/manage-resources-containers/
cloud_user_p_a5e85419@cloudshell:~ (playground-s-11-33c2cdd4) \$ helm upgrade my-nginx bitnami/nginx
Release "my-nginx" has been upgraded. Happy Helming!
NAME: my-nginx
LAST DEPLOYED: Thu Jun 11 07:12:40 2026
NAMESPACE: default
STATUS: deployed
REVISION: 2
DESCRIPTION: Upgrade complete
TEST SUITE: None
NOTES:
CHART NAME: nginx
CHART VERSION: 25.0.3
APP VERSION: 1.31.1

APP VERSION: 1.31.1

⚠ WARNING: Since August 28th, 2025, only a limited subset of images/charts are available for free.
Subscribe to Bitnami Secure Images to receive continued support and security updates.
More info at <https://bitnami.com> and <https://github.com/bitnami/containers/issues/83267>

** Please be patient while the chart is being deployed **

NGINX can be accessed through the following DNS name from within your cluster:

my-nginx.default.svc.cluster.local (port 80)

To access NGINX from outside the cluster, follow the steps below:

1. Get the NGINX URL by running these commands:

NOTE: It may take a few minutes for the LoadBalancer IP to be available.
Watch the status with: 'kubectl get svc --namespace default -w my-nginx'

```
export SERVICE_PORT=$(kubectl get --namespace default -o jsonpath="{.spec.ports[0].port}" services my-nginx)
export SERVICE_IP=$(kubectl get svc --namespace default my-nginx -o jsonpath='{.status.loadBalancer.ingress[0].ip}')
echo "http://${SERVICE_IP}:${SERVICE_PORT}"
```

WARNING: Rolling tag detected (bitnami/nginx:latest), please note that it is strongly recommended to avoid using rolling tags in a production environment.
+info https://techdocs.broadcom.com/us/en/vmware-tanzu/bitnami-secure-images/bitnami-secure-images/services/bsi-doc/apps-tutorials-understand-rolling-tags-containers-index.html
WARNING: Rolling tag detected (bitnami/git:latest), please note that it is strongly recommended to avoid using rolling tags in a production environment.
+info https://techdocs.broadcom.com/us/en/vmware-tanzu/bitnami-secure-images/bitnami-secure-images/services/bsi-doc/apps-tutorials-understand-rolling-tags-containers-index.html
WARNING: Rolling tag detected (bitnami/nginx-exporter:latest), please note that it is strongly recommended to avoid using rolling tags in a production environment.
+info https://techdocs.broadcom.com/us/en/vmware-tanzu/bitnami-secure-images/bitnami-secure-images/services/bsi-doc/apps-tutorials-understand-rolling-tags-containers-index.html

WARNING: There are "resources" sections in the chart not set. Using "resourcesPreset" is not recommended for production. For production installations, please set the following values according to your workload needs:

```
- cloneStaticSiteFromGit.gitSync.resources
- resources
```

+info https://kubernetes.io/docs/concepts/configuration/manage-resources-containers/

+info https://kubernetes.io/docs/concepts/configuration/manage-resources-containers/

cloud_user_p_a5e85419@cloudshell:~ (playground-s-11-33c2cdd4) \$ helm history my-nginx

REVISION	UPDATED	STATUS	CHART	APP VERSION	DESCRIPTION
1	Thu Jun 11 07:10:33 2026	superseded	nginx-25.0.3	1.31.1	Install complete
2	Thu Jun 11 07:12:40 2026	deployed	nginx-25.0.3	1.31.1	Upgrade complete

cloud_user_p_a5e85419@cloudshell:~ (playground-s-11-33c2cdd4) \$ helm rollback my-nginx 1

Rollback was a success! Happy Helming!

cloud_user_p_a5e85419@cloudshell:~ (playground-s-11-33c2cdd4) \$ helm history my-nginx

REVISION	UPDATED	STATUS	CHART	APP VERSION	DESCRIPTION
1	Thu Jun 11 07:10:33 2026	superseded	nginx-25.0.3	1.31.1	Install complete
2	Thu Jun 11 07:12:40 2026	superseded	nginx-25.0.3	1.31.1	Upgrade complete
3	Thu Jun 11 07:13:25 2026	deployed	nginx-25.0.3	1.31.1	Rollback to 1

cloud_user_p_a5e85419@cloudshell:~ (playground-s-11-33c2cdd4) \$ helm uninstall my-nginx

release "my-nginx" uninstalled

cloud_user_p_a5e85419@cloudshell:~ (playground-s-11-33c2cdd4) \$ helm list

NAME	NAMESPACE	REVISION	UPDATED	STATUS	CHART	APP VERSION
cloud_user_p_a5e85419@cloudshell:~ (playground-s-11-33c2cdd4) \$ kubectl get all						
NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE	
service/kubernetes	ClusterIP	34.118.224.1	<none>	443/TCP	7m49s	
service/my-nginx	LoadBalancer	34.118.239.231	35.238.156.5	80:31690/TCP,443:32278/TCP	3m31s	

cloud_user_p_a5e85419@cloudshell:~ (playground-s-11-33c2cdd4) \$

Task 23 — Terraform Fundamentals + Infrastructure as Code (IaC)

If Kubernetes is about managing applications, then Terraform is about managing infrastructure.

Solution :

- Open cloud shell, Run: terraform version, create project folder - mkdir terraform-lab & cd terraform-lab.
- Create terraform file, nano main.tf, paste this inside : (replace project id with yours)

```
terraform {
  required_providers {
    google = {
      source = "hashicorp/google"
      version = "~> 5.0"
    }
  }
}
```

```
provider "google" {
  project = "YOUR_PROJECT_ID"
  region = "us-central1"
}
```

Save it: Ctrl O + Enter + Ctrl X

- Initialize terraform - terraform init , Validate Configuration - terraform validate
- Add your first resource: nano main.tf, append this to the file (give unique bucket name) & save it:

```
resource "google_storage_bucket" "demo_bucket" {
  name     = "demo-bucket-12345"
  location = "US"
}
```
- Format file - terraform fmt, Plan - terraform plan
- Run - terraform apply, type yes.
- Destroy - terraform destroy, type yes.

Concepts :

Terraform enables Infrastructure as Code by defining cloud resources in configuration files rather than manually creating them through the console. Using commands such as terraform init, plan, and apply, infrastructure can be provisioned, version-controlled, and reproduced consistently across environments.

Terraform consists of:

Provider - Connects Terraform to a cloud. Example: Google Cloud, AWS, Azure, Kubernetes, Docker.

Resource - Actual thing being created. Examples: VM, Bucket, Firewall Rule, VPC

Understanding Terraform file -

Terraform Block - Defines required providers.

Provider Block - Tells Terraform: Use Google Cloud/Use this project/Use this region

Understanding Initialize terraform command - Terraform downloads: Google Provider Plugin

Similar to: npm install | pip install

Why Validate - Checks: Syntax/Provider Configuration/Basic Errors before creating anything.

Understanding adding first resource command - Terraform now understands: Desired State: Create Storage Bucket

Why formatting - In real projects: Code Reviews → Consistent Formatting Required

Understanding Terraform plan command - Terraform does NOT create anything yet. It simply says: Here's what I would do, Example: Plan: 1 to add/0 to change/0 to destroy

Sandbox screenshots –

```
cloud_user_p_d27290f2@cloudshell:~ (playground-s-11-7f856e0c)$ terraform version
Terraform v1.5.7
on linux_amd64

Your version of Terraform is out of date! The latest version
is 1.15.6. You can update by downloading from https://www.terraform.io/downloads.html
cloud_user_p_d27290f2@cloudshell:~ (playground-s-11-7f856e0c)$ mkdir terraform-lab
cd terraform-lab
cloud_user_p_d27290f2@cloudshell:~/terraform-lab (playground-s-11-7f856e0c)$ nano main.tf
cloud_user_p_d27290f2@cloudshell:~/terraform-lab (playground-s-11-7f856e0c)$ nano main.tf
cloud_user_p_d27290f2@cloudshell:~/terraform-lab (playground-s-11-7f856e0c)$ gcloud config get-value project
Your active configuration is: [cloudshell-17528]
playground-s-11-7f856e0c
cloud_user_p_d27290f2@cloudshell:~/terraform-lab (playground-s-11-7f856e0c)$ terraform init

Initializing the backend...

Initializing provider plugins...
- Finding hashicorp/google versions matching "~> 5.0"...
- Installing hashicorp/google v5.45.2...
- Installed hashicorp/google v5.45.2 (signed by HashiCorp)

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
cloud_user_p_d27290f2@cloudshell:~/terraform-lab (playground-s-11-7f856e0c)$ terraform validate
Success! The configuration is valid.
```

```
cloud_user_p_d27290f2@cloudshell:~/terraform-lab (playground-s-11-7f856e0c)$ nano main.tf
cloud_user_p_d27290f2@cloudshell:~/terraform-lab (playground-s-11-7f856e0c)$ terraform fmt
cloud_user_p_d27290f2@cloudshell:~/terraform-lab (playground-s-11-7f856e0c)$ terraform plan

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# google_storage_bucket.demo_bucket will be created
+ resource "google_storage_bucket" "demo_bucket" {
  + effective_labels      = (known after apply)
  + force_destroy         = false
  + id                   = (known after apply)
  + location              = "US"
  + name                  = "terraform-buck-987"
  + project               = (known after apply)
  + project_number        = (known after apply)
  + public_access_prevention = (known after apply)
  + rpo                   = (known after apply)
  + self_link             = (known after apply)
  + storage_class          = "STANDARD"
  + terraform_labels      = (known after apply)
  + uniform_bucket_level_access = (known after apply)
  + url                   = (known after apply)
}

Plan: 1 to add, 0 to change, 0 to destroy.
```

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you run "terraform apply" now.

```
cloud_user_p_d27290f2@cloudshell:~/terraform-lab (playground-s-11-7f856e0c)$ terraform apply
```

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

```
# google_storage_bucket.demo_bucket will be created
+ resource "google_storage_bucket" "demo_bucket" {
+   effective_labels      = (known after apply)
+   force_destroy         = false
+   id                   = (known after apply)
+   location              = "US"
+   name                 = "terraform-buck-987"
+   project              = (known after apply)
+   project_number       = (known after apply)
+   public_access_prevention = (known after apply)
+   rpo                  = (known after apply)
+   self_link            = (known after apply)
+   storage_class         = "STANDARD"
+   terraform_labels     = (known after apply)
+   uniform_bucket_level_access = (known after apply)
+   url                  = (known after apply)
+ }
}
```

Plan: 1 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?

Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

google_storage_bucket.demo_bucket: Creating...

google_storage_bucket.demo_bucket: Creation complete after 2s [id=terraform-buck-987]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

```
cloud_user_p_d27290f2@cloudshell:~/terraform-lab (playground-s-11-7f856e0c)$ terraform destroy
```

google_storage_bucket.demo_bucket: Refreshing state... [id=terraform-buck-987]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
- destroy

Terraform will perform the following actions:

```
# google_storage_bucket.demo_bucket will be destroyed
- resource "google_storage_bucket" "demo_bucket" {
-   default_event_based_hold = false -> null
-   effective_labels         = {} -> null
-   enable_object_retention  = false -> null
-   force_destroy            = false -> null
-   id                      = "terraform-buck-987" -> null
-   labels                  = {} -> null
-   location                = "US" -> null
-   name                    = "terraform-buck-987" -> null
-   project                 = "playground-s-11-7f856e0c" -> null
-   project_number          = 690355020677 -> null
-   public_access_prevention = "inherited" -> null
-   requester_pays          = false -> null
-   rpo                    = "DEFAULT" -> null
-   self_link               = "https://www.googleapis.com/storage/v1/b/terraform-buck-987" -> null
-   storage_class            = "STANDARD" -> null
-   terraform_labels        = {} -> null
-   uniform_bucket_level_access = false -> null
-   url                    = "gs://terraform-buck-987" -> null
-
-   soft_delete_policy {
-     effective_time      = "2026-06-12T11:58:26.157Z" -> null
-     retention_duration_seconds = 604800 -> null
-   }
- }
}
```

Plan: 0 to add, 0 to change, 1 to destroy.

Do you really want to destroy all resources?

Do you really want to destroy all resources?

Terraform will destroy all your managed infrastructure, as shown above.
There is no undo. Only 'yes' will be accepted to confirm.

Enter a value: yes

google_storage_bucket.demo_bucket: Destroying... [id=terraform-buck-987]

google_storage_bucket.demo_bucket: Destruction complete after 1s

Destroy complete! Resources: 1 destroyed.

Task 24 — Google Cloud Pub/Sub + Event-Driven Architecture

Up until now, almost everything you've built has been: User → Application → Response. This is called synchronous communication. But in large systems, services often don't talk directly to each other. Instead they communicate through messages. That's where Pub/Sub comes in.

Real-World Scenario - Imagine you're shopping on Amazon. You click: Place Order, What happens?

Not just one thing. Many things happen: Order Service → Payment Service → Inventory Service → Shipping Service → Email Service.

If every service waited for every other service: Slow | Fragile | Failure-prone.

Instead: Order Created → Message Published → Other Services React

Solution –

- Open Pub/Sub, Create topic, Name: orders-topic.
- Open orders-topic, Click create subscription, Name: email-service.
- Publish Message: Open orders-topic, click messages and then publish messages, message body: Order #1001 created.
- Pull Message: Open email-service, Click messages and then pull messages, you should see: Order #1001 created.
- Create second publisher: Create another subscription, Name: shipping-service. Publish another message: Order #1002 Created.
- Pull from shipping service: Click Pull messages, you'll also see: Order #1002 Created

Concepts –

Google Cloud Pub/Sub is a fully managed messaging service that enables asynchronous communication between distributed systems. Publishers send messages to topics, while subscribers consume messages independently, allowing applications to be loosely coupled, scalable, and fault tolerant.

Real-Life Analogy - Think of YouTube. Creator uploads video → Subscribers receive notification. Creator doesn't contact each subscriber individually.

Instead: Creator → YouTube → Subscribers | Pub/Sub works similarly.

Creating Topic - What Did We Create? A communication channel. Nothing consumes messages yet.

Think: Empty mailbox

Creating subscription - Topic receives messages. Subscription receives copies of messages.

Publishing Message - Publisher sent: Order #1001 Created to: orders-topic

Sandbox screenshots:



Topics					Learn	Show info panel
Create topic Delete Tags						
List Metrics						
Filter Filter topics						
Topic ID ↑	Encryption key	Topic name	Retention	Ingestion source		
orders-topic	Google-managed	projects/playground-s-11-3b56fbdd/topics/orders-topic	-	-		

←orders-topic

Edit

+ Trigger Cloud Run function

Import

Delete

Learn

Show info panel

Topic name

projects/playground-s-11-3b56fbbd/topics/orders-topic

Tags

Subscriptions

Snapshots

Metrics

Details

Messages

Test if your subscription can receive a manually published message

Step 1

Manually publish a message containing data and attributes with metadata to this topic

Publish message

Manually publish a message containing data and attributes with metadata to this topic.

Step 2

Select a Cloud Pub/Sub subscription to pull messages from

Select a Cloud Pub/Sub Subscription to pull messages from *

Subscriptions

Create subscription

Delete

Tags

Learn

Show info panel

List

Metrics

Filter

Filter subscriptions

	State	Subscription ID ↑	Delivery type	Topic name	Ack deadline	Retention	Message ordering	Exactly		
☐	✓	email-service	Pull	projects/playground-s-...	10 seconds	7 days	Disabled	Disabl	⋮	▼
☐	✓	orders-topic-sub	Pull	projects/playground-s-...	10 seconds	7 days	Disabled	Disabl	⋮	▼
☐	✓	shipping-service	Pull	projects/playground-s-...	10 seconds	7 days	Disabled	Disabl	⋮	▼

←orders-topic-...

Edit

Create snapshot

Replay messages

Purge messages

Detach

Delete

Learn

Show info panel

Subscription name

projects/playground-s-11-3b56fbbd/subscriptions/orders-topic-sub

Subscription state

✓ active

Topic name

projects/playground-s-11-3b56fbbd/topics/orders-topic

Tags

Metrics

Details

Messages

Click Pull to view messages and temporarily delay message delivery to other subscribers. Select Enable ACK messages and then click ACK next to the message to permanently prevent message delivery to other subscribers.

Pull

☐ Enable ack messages

Filter

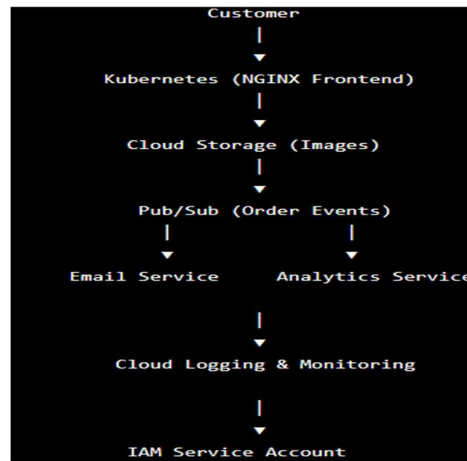
Filter messages

Publish time	Attribute keys	Message body	Ordering key	Ack ↑
Jun 14, 2026, 5:29:42 PM	—	Order #1001 Created	—	Deadline exceeded
Jun 14, 2026, 5:32:33 PM	—	Order #1002 Created	—	Deadline exceeded

FINAL TASK (Task 25)— End-to-End Cloud Native Mini Project (From Scratch)

Scenario - You have joined Accenture as a Cloud Native Developer. The client wants to migrate their Food Ordering website to Google Cloud. Your team is responsible for deploying the application.

Architecture -



Solution –

- Create Kubernetes cluster, Name: food-cluster, connect to cloud shell, verify: `kubectl get nodes`.
- Create cloud storage bucket, Name: food-images-<anythingunique>, upload any image inside it.
[Later your frontend would fetch this image.]
- Deploy frontend: `kubectl create deployment food-frontend --image=nginx`, verify: `kubectl get deployments`.
Check pods: `kubectl get pods`.
- Expose application: `kubectl expose deployment food-frontend --type=ClusterIP --port=80`, verify: `kubectl get svc`.
[You just deployed nginx inside Kubernetes, pretend this is your client website]
- Scale application: `kubectl scale deployment food-frontend --replicas=3`, verify: `kubectl get pods`.
[To handle more users]
- Simulate application update: `kubectl set image deployment/food-frontend nginx=nginx:1.26`,
watch rollout: `kubectl rollout status deployment/food-frontend`.
- Rollback: `kubectl rollout undo deployment/food-frontend`, verify: `kubectl rollout history deployment/food-frontend` [If prod deployment fails, rollback comes to the rescue]
- Create pubsub topic, Name: orders-topic, Create subscription: email-service
[Every time someone places an order, your application sends: Order Created to this topic]
Publish message from topic: Order #5001 Created, and pull it from subscription.
- Install helm: Verify: `helm version`, add repo: `helm repo add bitnami https://charts.bitnami.com/bitnami`, update:
`helm repo update`, install: `helm install demo bitnami/nginx`, Verify: `helm list`.
[helm replaces entire yaml work]
- Open monitoring, and just observe, observe logging too. [If prod crashes, engg investigate here]
- Create service account: food-app-sa
[Applications should authenticate using Service Accounts, not personal accounts]
- Cleanup: `helm uninstall demo`, `kubectl delete deployment food-frontend`, `kubectl delete service food-frontend`,
and finally delete the cluster.

Concepts: Deploying a cloud native application: We define infrastructure using Terraform, containerize the application with Docker, store images in Artifact Registry, deploy them to GKE, expose services through Kubernetes networking, manage deployments using Helm, scale workloads manually or automatically using HPA, use Pub/Sub for asynchronous communication between microservices, monitor applications using Cloud Logging and Cloud Monitoring, and secure access through IAM and service accounts.

Sandbox screenshots:

Kubernetes clusters

CreateDeployRefreshAttach clusterNew

Learn

OverviewUtilizationObservabilityCost Optimization

Health ⓘ
100% healthy

No recommendations

Upgrade ⓘ
100% up to date

No recommendations

Estimated monthly cost ⓘ
\$0.00 / month · 0%

No recommendations

Filter Enter property name or value ⓘ ⋮

☐ Status

Name ↑

Location

Number of nodes

Total vCPUs

Total memory

Notifications

Labels

☐

food-cluster

us-central1-a

1

2

2 GB

—

⋮

← Bucket details

Edit bucketGo to pathRefresh

food-images1

Location: us-central1 (Iowa)Storage class: StandardPublic access: Value hiddenProtection: Soft Delete

ObjectsConfigurationPermissionsProtectionLifecycleObservabilityInventory ReportsOperations

Folder browser

food-images1

Buckets > food-images1

Create folderUpload ▾Transfer data ▾Create batch operationsOther services ▾Learn

Filter by name prefix only ▾Filter Filter objects and foldersShow Live objects only ▾⋮

☐ Name

Size

Type

Create

☐

Azure VM Access Guide.docx

2.9 MB

application/vnd.openxmlformats-officedocument.wordprocessingml.document

Jul 2, 2026

⬇ ⋮

Topics

Create topicDeleteTags

ListMetrics

Filter Filter topics ⓘ ⋮

☐ Topic ID ↑

Encryption key

Topic name

Retention

Ingestion source

☐ [orders-topic](#)

Google-managed

projects/playground-s-11-a975380d/topics/orders...

—

—

⋮

← email-service

EditCreate snapshotReplay messagesPurge messagesDetachDelete

Subscription name

projects/playground-s-11-a975380d/subscriptions/email-service

Subscription state

active

Topic name

[projects/playground-s-11-a975380d/topics/orders-topic](#)

Tags

—

Metrics

Details

Messages

the message to permanently prevent message delivery to other subscribers.

Pull

☐ Enable ack messages

Filter Filter messages ⓘ ⋮

Publish time

Attribute keys

Message body

Ack ↑

Jul 2, 2026, 5:27:53 PM

—

Order #5001 Created

Ack

Subscriptions							+ Create subscription Delete Tags	
List	Metrics							
Filter Filter subscriptions ? ⋮								
☐	State	Subscription ID ↑	Delivery type	Topic name	Ack deadline	Retention		
☒	✓	email-service	Pull	projects/playground-s...	20 seconds	7 days	⋮	✓

```
Welcome to Cloud Shell! Type "help" to get started, or type "gemini" to try prompting with Gemini CLI.
Your Cloud Platform project in this session is set to playground-s-11-a975380d.
Use 'gcloud config set project [PROJECT_ID]' to change to a different project.
gcloud container clusters get-credentials food-cluster --zone us-central1-a --project playground-s-11-a975380dcloud_user_p_c064fa86@cloudshell:~ (playground-s-11-a975380d)$ gcloud container c
lusters get-credentials food-cluster --zone us-central1-a --project playground-s-11-a975380d
Fetching cluster endpoint and auth data.
Regional Access Boundary HTTP request failed after retries: response_data={'error': {'code': 404, 'message': 'Account not found for email: bb49f58bad@cloud_user_p_c064fa86@linuxacademygclabs.
com', 'status': 'NOT_FOUND'}}, retryable_error=False
kubeconfig entry generated for food-cluster.
cloud_user_p_c064fa86@cloudshell:~ (playground-s-11-a975380d)$ gcloud container clusters get-credentials food-cluster --region us-central1-a
Fetching cluster endpoint and auth data.
Regional Access Boundary HTTP request failed after retries: response_data={'error': {'code': 404, 'message': 'Account not found for email: bb49f58bad@cloud_user_p_c064fa86@linuxacademygclabs.
com', 'status': 'NOT_FOUND'}}, retryable_error=False
kubeconfig entry generated for food-cluster.
cloud_user_p_c064fa86@cloudshell:~ (playground-s-11-a975380d)$ kubectl get nodes
NAME                                STATUS    ROLES    AGE      VERSION
gke-food-cluster-default-pool-b4acffdc-3v7f    Ready    <none>    2m48s    v1.35.5-gke.1163012
cloud_user_p_c064fa86@cloudshell:~ (playground-s-11-a975380d)$ kubectl create deployment food-frontend --image=nginx
deployment.apps/food-frontend created
cloud_user_p_c064fa86@cloudshell:~ (playground-s-11-a975380d)$ kubectl get deployments
NAME      READY   UP-TO-DATE   AVAILABLE   AGE
food-frontend  1/1     1            1           10s
cloud_user_p_c064fa86@cloudshell:~ (playground-s-11-a975380d)$ kubectl get pods
NAME      READY   STATUS    RESTARTS   AGE
food-frontend-687b765b48-48tg8  1/1     Running    0          22s
cloud_user_p_c064fa86@cloudshell:~ (playground-s-11-a975380d)$ kubectl expose deployment \
--type=ClusterIP \
--port=80
service/food-frontend exposed
cloud_user_p_c064fa86@cloudshell:~ (playground-s-11-a975380d)$ kubectl get svc
NAME      TYPE        CLUSTER-IP    EXTERNAL-IP  PORT(S)    AGE
food-frontend  ClusterIP   34.118.230.55  <none>       80/TCP     8s
kubernetes    ClusterIP   34.118.224.1   <none>       443/TCP    11m
cloud_user_p_c064fa86@cloudshell:~ (playground-s-11-a975380d)$ kubectl scale deployment food-frontend --replicas=3
deployment.apps/food-frontend scaled
```

```
cloud_user_p_c064fa86@cloudshell:~ (playground-s-11-a975380d)$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
food-frontend-687b765b48-48tg8  1/1     Running    0          86s
food-frontend-687b765b48-q5jtz  1/1     Running    0          8s
food-frontend-687b765b48-ssz4r  1/1     Running    0          8s
cloud_user_p_c064fa86@cloudshell:~ (playground-s-11-a975380d)$ kubectl set image deployment/food-frontend nginx=nginx:1.26
deployment.apps/food-frontend image updated
cloud_user_p_c064fa86@cloudshell:~ (playground-s-11-a975380d)$ kubectl rollout status deployment/food-frontend
Waiting for deployment "food-frontend" rollout to finish: 1 out of 3 new replicas have been updated...
Waiting for deployment "food-frontend" rollout to finish: 1 out of 3 new replicas have been updated...
Waiting for deployment "food-frontend" rollout to finish: 2 out of 3 new replicas have been updated...
Waiting for deployment "food-frontend" rollout to finish: 2 out of 3 new replicas have been updated...
Waiting for deployment "food-frontend" rollout to finish: 2 out of 3 new replicas have been updated...
Waiting for deployment "food-frontend" rollout to finish: 2 out of 3 new replicas have been updated...
Waiting for deployment "food-frontend" rollout to finish: 1 old replicas are pending termination...
Waiting for deployment "food-frontend" rollout to finish: 1 old replicas are pending termination...
Waiting for deployment "food-frontend" rollout to finish: 1 old replicas are pending termination...
deployment "food-frontend" successfully rolled out
cloud_user_p_c064fa86@cloudshell:~ (playground-s-11-a975380d)$ kubectl rollout undo deployment/food-frontend
deployment.apps/food-frontend rolled back
cloud_user_p_c064fa86@cloudshell:~ (playground-s-11-a975380d)$ kubectl rollout history deployment/food-frontend
deployment.apps/food-frontend
REVISION  CHANGE-CAUSE
2         <none>
3         <none>

cloud_user_p_c064fa86@cloudshell:~ (playground-s-11-a975380d)$ helm version
version.BuildInfo{Version:"v4.2.0", GitCommit:"06468084e85c244c712834933d25ea232a4c2093", GitTreeState:"clean", GoVersion:"go1.26.3", KubeClientVersion:"v1.36"}
cloud_user_p_c064fa86@cloudshell:~ (playground-s-11-a975380d)$ helm repo add bitnami https://charts.bitnami.com/bitnami
"bitnami" has been added to your repositories
cloud_user_p_c064fa86@cloudshell:~ (playground-s-11-a975380d)$ helm repo update
Hang tight while we grab the latest from your chart repositories...
...Successfully got an update from the "bitnami" chart repository
Update Complete. *Happy Helming!*
```



```
cloud_user_p_c064fa86@cloudshell:~ (playground-s-11-a975380d)$ helm install demo bitnami/nginx
NAME: demo
LAST DEPLOYED: Thu Jul 2 11:59:17 2026
NAMESPACE: default
STATUS: deployed
REVISION: 1
DESCRIPTION: Install complete
TEST SUITE: None
NOTES:
CHART NAME: nginx
CHART VERSION: 25.0.12
APP VERSION: 1.31.2

⚠ WARNING: Since August 28th, 2025, only a limited subset of images/charts are available for free.
  Subscribe to Bitnami Secure Images to receive continued support and security updates.
  More info at https://bitnami.com and https://github.com/bitnami/containers/issues/83267

** Please be patient while the chart is being deployed **
NGINX can be accessed through the following DNS name from within your cluster:

    demo-nginx.default.svc.cluster.local (port 80)

To access NGINX from outside the cluster, follow the steps below:

1. Get the NGINX URL by running these commands:

NOTE: It may take a few minutes for the LoadBalancer IP to be available.
  Watch the status with: 'kubectl get svc --namespace default -w demo-nginx'

export SERVICE_PORT=$(kubectl get --namespace default -o jsonpath="{.spec.ports[0].port}" services demo-nginx)
export SERVICE_IP=$(kubectl get svc --namespace default demo-nginx -o jsonpath="{.status.loadBalancer.ingress[0].ip}")
echo "http://${SERVICE_IP}:${SERVICE_PORT}"
WARNING: Rolling tag detected (bitnami/nginx:latest), please note that it is strongly recommended to avoid using rolling tags in a production environment.
+info https://techdocs.broadcom.com/us/en/vmware-tanzu/bitnami-secure-images/bitnami-secure-images/services/bsi-doc/apps-tutorials-understand-rolling-tags-containers-index.html
WARNING: Rolling tag detected (bitnami/git:latest), please note that it is strongly recommended to avoid using rolling tags in a production environment.
+info https://techdocs.broadcom.com/us/en/vmware-tanzu/bitnami-secure-images/bitnami-secure-images/services/bsi-doc/apps-tutorials-understand-rolling-tags-containers-index.html
```

```
export SERVICE_PORT=$(kubectl get --namespace default -o jsonpath="{.spec.ports[0].port}" services demo-nginx)
export SERVICE_IP=$(kubectl get svc --namespace default demo-nginx -o jsonpath="{.status.loadBalancer.ingress[0].ip}")
echo "http://${SERVICE_IP}:${SERVICE_PORT}"
WARNING: Rolling tag detected (bitnami/nginx:latest), please note that it is strongly recommended to avoid using rolling tags in a production environment.
+info https://techdocs.broadcom.com/us/en/vmware-tanzu/bitnami-secure-images/bitnami-secure-images/services/bsi-doc/apps-tutorials-understand-rolling-tags-containers-index.html
WARNING: Rolling tag detected (bitnami/git:latest), please note that it is strongly recommended to avoid using rolling tags in a production environment.
+info https://techdocs.broadcom.com/us/en/vmware-tanzu/bitnami-secure-images/bitnami-secure-images/services/bsi-doc/apps-tutorials-understand-rolling-tags-containers-index.html
WARNING: Rolling tag detected (bitnami/nginx-exporter:latest), please note that it is strongly recommended to avoid using rolling tags in a production environment.
+info https://techdocs.broadcom.com/us/en/vmware-tanzu/bitnami-secure-images/bitnami-secure-images/services/bsi-doc/apps-tutorials-understand-rolling-tags-containers-index.html

WARNING: There are "resources" sections in the chart not set. Using "resourcesPreset" is not recommended for production. For production installations, please set the following values according to your workload needs:
- cloneStaticSiteFromGit.gitSync.resources
- resources
+info https://kubernetes.io/docs/concepts/configuration/manage-resources-containers/
cloud_user_p_c064fa86@cloudshell:~ (playground-s-11-a975380d)$ helm list
NAME      NAMESPACE    REVISION    UPDATED                               STATUS    CHART          APP VERSION
demo     default      1           2026-07-02 11:59:17.669991604 +0000 UTC deployed  nginx-25.0.12  1.31.2
cloud_user_p_c064fa86@cloudshell:~ (playground-s-11-a975380d)$ helm uninstall demo
release "demo" uninstalled
cloud_user_p_c064fa86@cloudshell:~ (playground-s-11-a975380d)$ kubectl delete deployment food-frontend
deployment.apps "food-frontend" deleted from default namespace
cloud_user_p_c064fa86@cloudshell:~ (playground-s-11-a975380d)$ kubectl delete service food-frontend
service "food-frontend" deleted from default namespace
```

===== THE END =====